

address reference, since the base table follows the save area.

Register 14 contains the return address for subprograms and the address of branch target instructions during the execution of branches.

Register 15 contains the entry point address for subprograms as they are called by this program module.

10.7.9. Summary

The 360 system architecture is based on a set of 16 general purpose registers and a set of register-register or register-memory data operations. A typical memory reference instruction carries a displacement and two register references. The displacement is limited to 4095 bytes, one of the registers depends on the absolute program loading address fixed just prior to execution, and the remaining register can be used for array indexing.

A compiler generates an *object module*, consisting of a TXT file containing instructions, constants and partially fixed-up addresses, an ESD table that specifies symbolic information for linkage to other object modules, and an RLD table that specifies how addresses are to be fixed up by the loaders. The nature of the machine is such that a program module must remain in the memory partition to which it was assigned throughout its execution.

For maximum run-time efficiency, the allocation of registers as array indices, base registers and data registers, and their dynamic assignment as such is important. Data access usually requires a *base address table* containing addresses into selected locations in data memory. The base addresses are separated by at least 4096 bytes, since any location within that range can be accessed through the base address and a displacement in the instruction.

Subroutine calls are governed by system conventions in which every subroutine must provide a save area, whose address is passed in register 13. The called subroutine is expected to save all but register 13 (but none of the floating-point registers), and restore these upon return. Other save area entries facilitate debugging and the tracing of subroutine calls upon an abnormal program termination. The *save area* system can be adapted to recursive procedure calls by dynamically allocating a new save area on each call and deallocating it upon a return.

The 360 system is probably the most widely used computer system in the world, despite certain shortcomings as a time-shared multiprogramming system. Its architecture has also been copied in non-IBM machines (e.g., Interdata and Amdahl), in order to exploit 360 software. It will undoubtedly continue to be among the major computer systems in the world for many years.

Exercises

1. How suitable is the IBM 360 architecture as an AOC machine simulator? Sketch a plan for support of a stack, variable addressing and procedure calls that exploit the machine's registers and addressing modes. Your system must be relocatable, or it will not be executable on a time-sharing 360.
2. When a 360 program begins execution, it is assigned to a memory partition that cannot be changed during its execution. Why not? What conventions must be satisfied by every program in order that the system could move programs about in memory arbitrarily during execution?
3. A save area is part of the caller's code, but the callee fills the save area to save registers, and later accesses it to restore the registers before returning. Is this a reasonable plan? Why or why not?
4. Compare the object module system for the IBM/360 with the USL structure for the HP-3000. What differences are dictated by the architectural differences? In what ways are the capabilities different?
5. Sketch an algorithm for the assignment of a limited number of registers as memory data pointers (see figure 10.27 and the associated discussion) and their management during execution. Strive for a reasonably efficient run-time implementation.
6. Sketch a Pascal system similar to Wirth's scheme for the CDC 6400 that exploits the 360 architecture. Impose whatever reasonable limitations on procedure nesting depth or expression temporaries, etc., that seem reasonable; however, avoid any limitation on the maximum size of a program if possible.
7. What changes would you make in the 360 architecture that would better suit the machine for time-sharing purposes? For the support of block-structured languages? For the support of Fortran?

10.8. A Generalized Code Generator

As the preceding machine examples indicate, the generation of code for some target machine by a compiler can require highly specialized algorithms tailored to the particular target machine. A number of alternative approaches to compiler construction have been proposed that attempt either to generalize code generation or to isolate the machine peculiarities in some fashion, in order to reduce the effort required to implement languages on more than one target machine.

A promising approach to more abstract code generation has recently been reported by Glanville and Graham (Glanville [1978]). It does not deal with all the issues of machine dependence and machine idioms, but does represent a significant contribution to the problem of selecting a suitable sequence of object code instructions, given an abstract syntax tree. The method has the virtues of generality and correctness. The target machine instruction set is represented as a table, and if the table is correct, the generator will either block or generate correct code.

The method requires an abstract syntax tree (AST) and a syntax-directed translation scheme (SDTS) as input. The SDTS is a description of the machine's instruction set and the arithmetic operations expressed in prefix notation. The AST is first translated to prefix by scanning its nodes in preorder. The prefix form of the AST may then be parsed by a modified LR parser, to yield a sequence of SDTS production numbers; the output sequence then represents the desired object code instruction sequence.

The SDTS underlying grammar will in general be ambiguous. This means that a deterministic LR parser can only be constructed by arbitrarily resolving its inadequate states in favor of one reduction or another, or a push action. It is essential that an arbitrary resolution of an inadequate state not result in a machine block on some input string. We shall develop necessary conditions on the SDTS that the machine not block.

The resolution of an inadequate state that provides two or more reduction possibilities should in general be toward the largest of the possible productions. As a result, the system selects instructions that encode the largest parts of the AST and therefore minimizes the number of generated instructions. Although the method cannot be proven to yield optimal code, some experiments have shown that the quality of its generated code is very good, with no attempt to improve the code quality by any special means.

The method does not directly deal with questions of register allocation, base register movement, etc. These matters should be handled with auxiliary algorithms that can interact with or accept code generated by this generalized system.

Example

A small set of machine instructions that have the general form of an SDTS grammar (chapter 7) is given below.

- | | |
|--|-------------------------|
| 1. $r.2 ::= (+\uparrow + k.1 \ r.1 \ r.2)$ | "add $r.2, k.1, r.1$ "; |
| 2. $r.1 ::= (+r.1\uparrow + k.1 \ r.2)$ | "add $r.1, k.1, r.2$ "; |
| 3. $r.1 ::= (+\uparrow k.1 \ r.1)$ | "add $r.1, k.1$ "; |
| 4. $r.1 ::= (+r.1\uparrow k.1)$ | "add $r.1, k.1$ "; |
| 5. $r.1 ::= (+r.1 \ r.2)$ | "add $r.1, r.2$ "; |
| 6. $r.2 ::= (+r.1 \ r.2)$ | "add $r.2, r.1$ "; |

7. $\lambda ::= (:= \uparrow + k.1 \ r.1 \ r.2)$	"store r.2,*k.1,r.1";
8. $\lambda ::= (:= + k.1 \ r.1 \ r.2)$	"store r.2, k.1, r.1";
9. $\lambda ::= (:= \uparrow k.1 \ r.1)$	"store r.1,*k.1";
10. $\lambda ::= (:= k.1 \ r.1)$	"store r.1,k.1";
11. $\lambda ::= (:= r.1 \ r.2)$	"store r.2,r.1";
12. $r.2 ::= (\uparrow + k.1 \ r.1)$	"load r.2,k.1,r.1";
13. $r.2 ::= (+ k.1 \ r.1)$	"load r.1,=k.1,r.1";
14. $r.2 ::= (+ r.1 \ k.1)$	"load r.2,=k.1,r.1";
15. $r.2 ::= (\uparrow r.1)$	"load r.2,*r.1";
16. $r.1 ::= (\uparrow k.1)$	"load r.1,k.1";
17. $r.1 ::= (k.1)$	"load r.1,=k.1";

The symbol to the left of "::<=" represents the destination of the result of a unit computation, and the prefix expression to its right describes the computation. The parentheses are metasymbols introduced for readability. Symbol λ indicates that there is no resulting register value; the instruction is executed solely for its side effects.

Symbol "r" represents a general purpose register, and "k" denotes a literal constant, typically an address offset. The store operator is "::<=" and the contents operator is $\uparrow$. The symbols r.1, r.2, k.1, etc. are SDTS nonterminals. If r.1 appears in the left member of a translation rule, then it corresponds to r.1 in the right member. Symbols r.1 and r.2 then possibly represent distinct registers. Thus the first ADD instruction (rule 1) adds the value in the memory location designated by k.1 and the contents of register r.1, leaving the result in register r.2.

Commutativity of operations is explicitly indicated by separate translation rules, e.g., rules 3 and 4.

Semantic restrictions can be imposed during the parsing process on instruction descriptions, permitting a greater number of special instructions to be incorporated in the translation rules. A single syntactic instruction pattern may then correspond to more than one instruction sequence. Thus two identical patterns (e.g., add r.1,r.2 and add r.2,r.1) could appear because of commutativity. Also one instruction could be a special case of another (e.g., "inc r.1" and "add r.1,k.1").

The source language must first be reduced to an intermediate representation (IR) that is a sentence in the underlying grammar of the instruction set SDTS. The IR may be generated from an AST by scanning it in preorder to yield a prefix expression. For example, the statement

$$A := B + C$$

might translate to the IR expression

$$:= + k.a \ r.7 + \uparrow + k.b \uparrow r.7 \uparrow k.c$$

where a,b,c designate constants and r.7 is a local base register. In this

translation, we have obviously assigned memory to the variables A, B, and C, so that the AST translation will appear in register notation.

The translation of an IR sentence to a sequence of machine instructions is carried out by an LR(1)-like deterministic parser. The parser is constructed from the target machine description, replacing λ by X and adding rules $S \rightarrow Z, Z \rightarrow ZX \mid \epsilon$ to the grammar.

The code-emitting semantics implied by the SDTS translation rules are handled essentially as described in chapter 7. For example, when a shift is performed with the input symbol r standing for a register, the specific register represented by that r is pushed onto the semantic stack. Then when a reduce is done, the semantic information necessary to generate a final machine instruction is available in the semantics stack.

The instruction to be used when a reduce action contains more than one instruction (inadequate state) is picked by a simple heuristic. Instructions are ordered by the table constructor into a "best instruction first" sequence. At code generation time, the instructions in the set for a specific reduce action are tested in that order until one is found that is semantically compatible with the information in the semantics stack. Since all instructions in a reduce set have the same instruction pattern, the cheapest instructions (according to some cost criteria) are tested first. It is possible for some instruction in the reduce action state to not be selected, owing to semantics restrictions. For example, an "add immediate" could be chosen if the immediate constant is sufficiently small, otherwise, a more general (and more costly) instruction must be selected.

Register allocation must occur as a subtask to the parsing action. After an instruction pattern has been semantically verified, and the result is non- λ , and if the result register is not semantically linked to some other register in the instruction pattern, then the register allocator will be requested to provide a free register of the appropriate class, marking it "used". If the result register r is semantically linked to a register in the instruction pattern, then r must be used as the target register and marked "used". Recall that the result register is always one of the operand registers in the IBM 360 architecture, but need not be in the CDC 6000 architecture.

As an example of a generated code sequence, the input expression

$$:= + k.a \ r.7 + \uparrow + k.b \uparrow r.7 \uparrow k.c$$

yields the code sequence

```
LOAD  r1,r7
LOAD  r2,c
ADD   r2,b,r1
STORE r2,a,r7
```

using the SDTS given above. (This example is worked out in detail in Glanville's paper.)

10.8.1. Table Construction

The table constructor first treats the underlying grammar as a context-free grammar and constructs the set of LR(0) states, ignoring inadequate states for the moment. There will be numerous inadequate states in general, and they will arise both from finite lookahead and from grammar ambiguity. The construction algorithm seeks to resolve these conflicts in favor of short instruction sequences—shift-reduce conflicts are resolved in favor of shifting. For the most part, more powerful instructions are used as more of an expression is shifted into the stack before an instruction covers it. Reduce-reduce conflicts are often resolved by the semantic restrictions; if not, the longest instruction is used. The parser clearly must be capable of identifying reduce-reduce conflicts during parsing and refer their resolution to a semantic resolver. In resolving shift-reduce conflicts, SLR(1) lookahead information is used to ensure that reduce actions are included where needed.

Given a general context-free grammar, the conflict-resolution rules would not necessarily yield a recognizer for the entire language generated by the grammar. However, the construction is language-preserving for uniform instruction sets, defined as follows.

Uniform Instruction Set

Let Σ designate the vocabulary of the instruction set description. An instruction set is said to be *uniform* if it satisfies the condition:

Any left (right) operand of a binary operator b is a valid left (right) operand of b in any prefix expression of Σ^* containing b . Any operand of a unary operator u is a valid operand of u in any prefix expression of Σ^* containing u . An instruction set is uniform if its description is uniform.

The essential idea of uniformity is that the operands of an operator must be valid independent of their context.

For example, consider the SDTS given above. The operands of $:=$ are all either registers (rules 10, 11) or special cases of registers (rules 7, 8, 9). The operands of the left-most $+$ in rules 1 to 6, 13, 14 are all either registers (rules 5, 6) or prefix expressions that become registers by rules 12, 16, or 17. Since a register is an argument of $\uparrow$ (by rule 15) and of $+$ (by rule 5), the arguments of the other occurrences of $+$ and $\uparrow$ represent special cases (i.e., ambiguities). Thus whenever $:=$, $+$, and $\uparrow$ occur, their operands are any prefix expressions corresponding to registers.

An example of an instruction set that is not uniform is

$$\begin{aligned} r &::= (+ \uparrow k k) \\ r &::= (+ k \uparrow k) \\ \lambda &::= (:= r r) \end{aligned}$$

Here, k is a valid first operand of $+$ only if the second operand is $\uparrow k$ and vice-versa.

Glanville shows how the uniformity condition can be verified by the parser constructor algorithm.

Given uniformity, an SLR(1) code generator can be constructed that can be shown to fail to reach its ACCEPT state in only one of the following two cases:

1. The code generator loops infinitely.
2. The input IR is not syntactically within the sequence of prefix expressions described by the instruction set.

Looping can occur through a recursive sequence of single production reductions, e.g., $A \Rightarrow B \Rightarrow C \Rightarrow \dots \Rightarrow A$, which can occur in practice if simple register-to-register moves are permitted. The potential of looping is easily tested in the finite state control and can be broken by a kind of “state-splitting” of certain of the generated states.

We need to consider the possibility of blocking—which would correspond to a syntax error if the input source were not generated internally from an AST.

The reduce action generates code by checking the semantic qualifications of the matched instruction pattern against the semantic qualifications indicated on the stack. If the set of instructions associated with a given reduce action are all semantically constrained, it is possible that none of the instructions will be compatible with the semantics on the stack. In this case, the code generator is said to *semantically block*. Semantic blocking can be avoided by the use of a default list of short instructions that contain no semantic restrictions and that together compute the desired expression. For example, consider a memory add-to-add instruction

madd k.2,k.1

This instruction can be encoded as the translation rules

$$\lambda := (:= k.1 + \uparrow k.1 \uparrow k.2), \text{ “madd k.1,k.2”}$$

$$\lambda := (:= k.1 + \uparrow k.2 \uparrow k.1), \text{ “madd k.2,k.1”}$$

The basic instruction pattern is “ $:= k + \uparrow k \uparrow k$ ”. If this pattern occurs in the IR, but the constants associated with each of the three k ’s are distinct, then the instruction cannot be used. The cure to this problem is to introduce new instructions with the patterns

$$\begin{aligned} r &::= \uparrow k \\ r &::= + \uparrow k \ r \\ \lambda &::= := k \ r \end{aligned}$$

that could be issued instead, simulating a non-restricted memory-to-memory add. The code generation could then proceed with these as though the longer instruction had been issued.

Glanville's table constructor constructs default instruction lists for all reduce actions having no semantically unrestricted instructions. The lists are obtained by simulating the action of the coder using as input the right hand side of the semantically restricted instruction and using only those instructions that are shorter than the one under consideration.

In the presence of semantically restricted instruction patterns (of which the table constructor has no knowledge), the code generator will choose one of a list of instructions in its reduce action. Assuming that this list has already been sorted by the table constructor, the instructions will be tested sequentially until one is found that is semantically compatible. In the event that no instruction is acceptable, a default list of substitutable instructions must be provided to implement that computation.

There are several classes of semantic restrictions that may have to be satisfied. Constants in the IR input may have to be equal to specific values, such as a 1 in an increment instruction, and logical registers may have to be equivalent to specific, actual registers. Multiple occurrences of a symbol in any class may have to refer to the same value or register. If the result is to appear in a register, then there must not be any references to the value in that register outside of that instruction pattern. Such references could exist only if some sort of common subexpression elimination has been done (see chapter 11). Finally, any additional semantic restrictions required to properly describe a particular target computer may be added to the code generator. The proof of correctness only requires that a default instruction or list of instructions be available for each semantically restricted instruction, so that it will always be possible to generate code.

Glanville shows that if looping and semantic blocking are eliminated by the extensions described above, and if the instruction description accurately describes the target machine, then the code generator produces correct code for all well-formed input.

The input IR is *well-formed* if

1. The input is a sequence of prefix expressions.
2. The operators and operands of the IR are from the same set as the operators and operands of the instructions and have the same meaning.
3. The sequence of prefix expressions is *valid*; that is, it is in the language generated or described by the instruction set.

Condition (3) must, of course, be checked by the implementor. The implementor can either prove that the routines generating the IR meet specification 1 or provide a simple routine to test the input to the code generator. It turns out that if the instruction set is uniform, condition 3 is also relatively easy to check.

10.8.2. Experimental Results

Several target computer descriptions, including the IBM 370 and the PDP-11 were input to a table constructor implemented in Pascal. Pascal

programs were manually translated into IR and input to the code generator. The PDP-11 code quality is competitive with the compiler "C" (Kernighan [1978]), yet uses no prior or post optimizations at all. Since the code generator produces assembly code, any postoptimizer can be used to further improve the code. On the PDP-11, some improvement can be obtained by exploiting short jump instructions and keeping track of register contents, for example. Similar results were found for the 370 code generator.

10.9. Bibliographical Notes

A good review of the uses of quads and triples in compiler construction and code improvement is given in Gries' text, "Compiler Construction for Digital Computers", Gries [1971], especially chapters 11, 13, 17, and 18. General reviews of code generation and the relation of machines to languages are given in four articles by Waite [1974a,b,c,d,e]. Poole [1974] discusses some compiler portability issues, including the use of intermediate languages and standard abstract machine descriptions.

Most of the material on the HP-3000, the CDC 6000 and the IBM System/360 is derived from manufacturer's manuals. The following is a minimum list:

- HP-3000 Computer System Reference Manual, part no. 03000-90019, Hewlett-Packard Co., 1501 Page Mill Rd., Palo Alto, CA., 94304.
- Control Data 6400/6500/6600 Computer Systems Reference Manual, Pub. No. 60100000, Control Data Corporation, 8100 34th Ave. So., Minneapolis, Minn. 55420.
- IBM System/360 Principles of Operation, Form A22-6821-3, IBM Corporation, Customer Manuals, Dept. B98, PO Box 390, Poughkeepsie, N.Y., 12602.
- IBM System/360 Operating System Assembler Language, Form C28-6514-4.
- IBM System/360 Supervisor and Data Management Services, Form C28-6646-2.
- IBM System/360 Operating System FORTRAN IV(G) Compiler Program Logic Manual, Form GY28-6638-1.

In addition, any technical library will contain textbooks on System/360 principles and programming. A good introductory treatment of the 360 architecture, assembler, loader conventions and macro processor is Donovan's text "Systems Programming", Donovan [1972].

Wirth's implementation of Pascal on the CDC 6400 is in his paper, "The Design of a Pascal Compiler," Wirth [1971a]. Glanville and Graham's work on a generalized code generator is in Glanville [1977].

An Algol 60 implementation on the IBM 360 is described in Bauer [1968].

OPTIMIZATION

The term *optimization* in a compiler is applied to any special algorithm designed to yield more efficient object code than would be obtained by a simple, straight-forward code generator. An optimization may replace a simple coding algorithm by a more sophisticated one, or it may be an operation on some compiler data structure that transforms the structure into an equivalent but more efficient one.

The goals of optimization are the reduction of execution time and the reduction of code and data space. These two objectives are often found (unfortunately) to conflict. For example, execution time can often be reduced to some extent by adding more instructions. Thus a procedure call might be replaced by its code, thereby saving the execution time required to call a procedure. Ideally, a compiler user should be able to specify whether he wishes a minimum number of instructions or a minimum execution time, however this is seldom possible. Usually, some compromise is possible, and often a reduction in amount of code also results in a reduction of execution time as well.

Optimization is usually desirable because a simple code generator may fail to fully exploit the algebraic properties of the source language, or the full potential of the target machine's instruction set. For example, a direct translation of a statement like

$$A := 1 + B + 6;$$

would emit two additions. An optimization that exploits associativity of addition would reduce the translation to one addition. Also, a statement of the form

$$I := I + 1;$$

is very common in source programs. A general translation on a stack machine would yield the code

```
LOAD  I;  
LOAD  =1;  
ADD;  
STOR  I;
```

Now many machines have “increment memory” instructions, so that the statement $I := I + 1$ could be translated into

INCM I;

instead.

An optimization may be classified as *machine-dependent* or *machine-independent*.

Machine-dependent optimizations stem from special machine properties that can be exploited to reduce the amount of code or execution time.

Machine-independent optimizations depend only on arithmetic properties of the operations in the language and not on peculiarities of the target machine. Some common machine-independent optimizations are

- Constant arithmetic subsumption or folding, i.e., operations on constant expressions performed by the compiler rather than emitted as code.
- Identification and removal of identity or null operations. e.g., adding zero, multiplying by 1.
- Rearrangement of expression trees, exploiting commutativity, associativity or distributivity of certain operators, with the objective of reducing the number of operations.
- Identification of common subexpressions, for possible one-time evaluation.
- Elimination of useless or unreachable code.
- Movement of code from inside a loop to outside the loop, possibly changing its form to preserve the program semantics.

These are listed roughly in order of increasing complexity of their algorithms. Thus (1) and (2) are relatively easy to implement on an AST, (3) and (4) are somewhat harder, and (5) requires a control flow analysis of the loop statements.

We shall not attempt to develop or even classify all the possible optimizations. Nor will we present a systematic approach to optimization. We will discuss some of the most commonly used optimizations in sufficient detail for them to be useful in most compilers. Beyond that, the reader must refer to the voluminous literature on the subject.

11.1. Machine-dependent Optimizations

The nature of a machine-dependent optimization depends heavily on the target machine and the source language. A machine-dependent optimization may be applied to the AST by identifying special subtrees that happen to fit a machine feature, or it may be applied to an object code sequence. Still other optimizations deal with register and memory allocation and arrange that data appear in the appropriate registers in a reasonably optimal manner.

Some examples of machine-dependent optimizations are

1. *Register allocation.* In a typical computer, the most efficiently accessed memory is the most scarce resource (e.g., one or a few arithmetic registers, or the top-of-stack registers). The least efficiently accessed memory is the most plentiful (e.g., solid state memory, disk, drum, or tape). Most operations can be performed only on data in registers, and the consequent movement of data between registers and mass memory is a bookkeeping maneuver that contributes nothing directly to the computation. Hence the allocation of the scarce resources to achieve high efficiency in program execution is an important optimization.
2. *Special machine features.* Some common machine features (or *idioms*, a term first applied by Hall [1974]) that can potentially be exploited include: immediate instructions (a value is part of the instruction), incrementation (a memory location can be incremented by some constant), use of indexing or indirection, vector operations, etc.
3. *Data intermixed with instructions.* On many machines, data can be more efficiently accessed if it is intermixed with the instruction sequence. Some optimal arrangement of instructions and data therefore exists.

11.2. Machine-independent Optimizations

These optimizations are based on the mathematical properties of a sequence of source statements. An optimization essentially amounts to an analysis of the overall purpose of the statement sequence, then finding an equivalent sequence that will translate to the least amount of code.

Thus constant arithmetic can always be done by the compiler, rather than by emitting instructions to perform the arithmetic during execution. Arithmetic properties of the operations may also be exploited in the search for minimal code; their use might uncover some additional constant arithmetic, or some common subexpressions.

Whether an arithmetic property may be exploited depends on the specification of the language. If the language quite clearly specifies that all operations are to be performed from left to right in exactly the sequence dictated by the form of the expression, then arithmetic optimization is clearly limited. On the other hand, many modern languages specify that the operations may be performed in any order consistent with the arithmetic meaning of the expression.

Of the several arithmetic properties of the real numbers, the identity and null properties can always be exploited to reduce emitted code. Commutativity and associativity are very useful in subsuming constants, in identifying common subexpressions, and in optimizing register allocation. Distributivity is potentially useful in constant subsumption and subexpression recognition, but is difficult to exploit in a general way. The language may prohibit the use of distributivity for optimization—it may require that a parenthesized expression be performed first and not under any circumstances distributed among outer operations.

Computer arithmetic technically fails to satisfy any of the three properties of commutativity, associativity and distributivity, as the following three examples indicate:

Failure of Commutativity

Consider the statement

$$I := I + F(A);$$

where $F(A)$ is a function call that happens to change the value of I as a side effect. (I must be in a domain that is accessible to the function F .) On a stack machine, in the order shown, the emitted code might be

```
LOAD I;    {current value}
LOAD A;    {the parameter for the function call}
CALL F;
ADD;       {the change in I does not affect the copy
           previously written on the stack}
STOR I;
```

In commuted order, we would have instead:

```
LOAD A;
CALL F;
LOAD I;    {the new value}
ADD;
STOR I;    {a different result}
```

This side effect of the function call stems from an undesirable property of functions in several common languages, their ability to access and modify variables that appear with them in arithmetic expressions. For this reason, certain modern languages restrict functions to access only those variables local to the function call. Then this side effect could not exist; the compiler would be able to determine from the replacement statement alone whether the function could affect any other variables.

Failure of Associativity

Consider the following statements implemented on a machine with integer arithmetic in the range -32768 through 32767 (16-bit twos complement integers):

```
I:=30000;
J:=20000;
K:=21000;
I:=I-J+K;
```

If the last statement is executed in left-to-right order, the result 31000 is obtained without overflow. However, if the statement were instead

```
I:=I+K-J;
```

then the addition would overflow before the subtraction is performed.

Here, if the language specification permits associativity for the sake of optimization, the programmer is at fault in not considering the possibility of overflow.

Failure of Distributivity

Consider the following example:

```
I:=15;
J:=4000;
K:=3000;
I:=I*(J-K);
```

The last statement, if performed in the order indicated by the parenthesizing, will yield 15000 without overflow. However, the distributive equivalent

```
I:=I*J - I*K;
```

will overflow twice.

Other sources of failure of computer arithmetic to satisfy mathematical properties stem from (1) integer division, where the fractional part is truncated to yield the largest integer less than the quotient, and (2) floating-point subtraction, where the two numbers are very nearly alike; the result will likely have lost all significance.

Despite these difficulties, algebraic optimization is often provided by a compiler for those languages that permit it. Where optimization is provided, the programmer must be alert to the potential failure of his coded algorithm; the optimizing compiler may generate a somewhat different algorithm than he coded.

11.2.1. Expression AST Optimizations

Some arithmetic optimizations are most effectively performed on an abstract-syntax tree for an arithmetic expression. Let us consider some of these, for a set of operations that includes the four binary arithmetic operations and unary minus. A similar principle applies to binary and unary logical operations.

The easiest optimizations are those based on the identity and null properties of 0 and 1. We need only inspect the environment of a constant 0 or 1 in the AST and apply one of the operations indicated in figure 11.1. Here, T stands for some subtree of arithmetic expressions. These obviously express the identities

$$\begin{aligned} 0 + T &= T + 0 = T \\ T - 0 &= T \\ 0 - T &= -T \\ 0 * T &= T * 0 = 0 \\ 0 / T &= 0 \quad \{\text{however, } 0/0 \text{ is undefined}\} \\ T / 0 &= \infty \quad (\text{or overflow}) \\ 1 * T &= T * 1 = T / 1 = T \end{aligned}$$

Such expressions are not likely to be written directly by a knowledgeable programmer, but frequently arise through the use of macros with parameters. Without compiler optimization, the macro definition must contain many special tests in order to yield reasonably optimal code.

Constant evaluation is facilitated by a rearrangement of the AST designed to bring constants together. For example, the expression

$$4 * 5 + I - 6$$

has the AST shown in figure 11.2. In this form, only the $4 * 5$ evaluation is possible. However, associativity may be applied to the $+$ and $-$ to yield the AST shown in figure 11.3; the multiplication and subtraction may then be

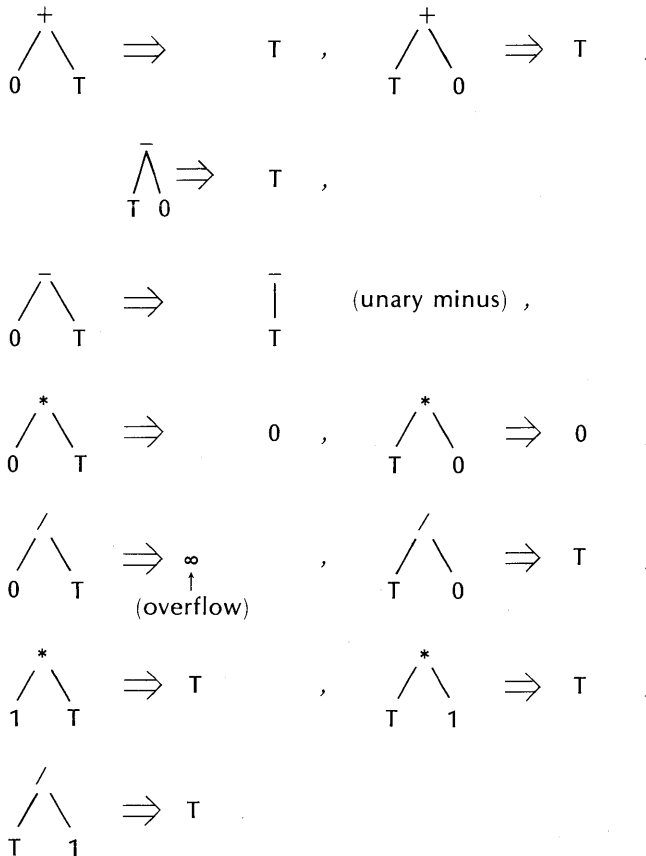


Figure 11.1. Abstract-syntax tree (AST) optimizations based on arithmetic properties of 0 and 1.

performed to yield the the tree for $14 + I$. A general application of commutativity and associativity in an AST is called *flattening*.

11.2.2. Flattening

In general, a set of connected $+$ and $-$ nodes may be collected into a single node with more than two children; the result is to *flatten* an AST. The flattened tree then facilitates the recognition of optimizations through associativity and commutativity.

An *addition cluster* of nodes in an AST is a set of nodes A such that (1) every member of A is in $\{+, -\}$, and (2) the set A forms a tree. Then a cluster A may be reduced to a single node by applying the transformations shown in figure 11.4. For example, a binary $-$ node can be replaced by a

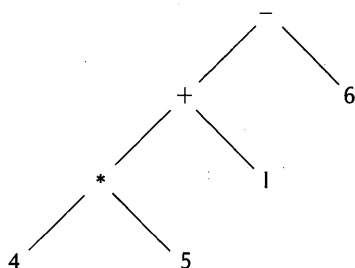


Figure 11.2. AST for an expression $4*5+1-6$.

binary $+$ node by negating the second child of the $-$ node. The overall effect of the transformations of figure 11.4 is to obtain as large a cluster of $+$ nodes near the top of a tree as possible, with unary $-$ nodes pushed down as much as possible.

A cluster of $*$ and $/$ nodes can be formed through similar transformations. A binary division A/B can be replaced by $A*(1/B)$, provided that the division is performed in floating-point and not fixed-point.

Once a cluster of additions or multiplications is identified, the AST may be flattened by merging each cluster into a single node, with several children. Thus the expression

$$A*B*C/D + E*F - D*A$$

has the flattened tree shown in figure 11.5. The multiple children of some $+$ or $*$ node may then be rearranged in any manner to achieve an optimization. For example, the constant children can be placed to the right (or left) of variable children and subsumed into a single constant node.

Parenthesized expressions must be protected (in most languages) from cluster merging. Thus the tree for $A+(B+C)$ cannot be flattened into a

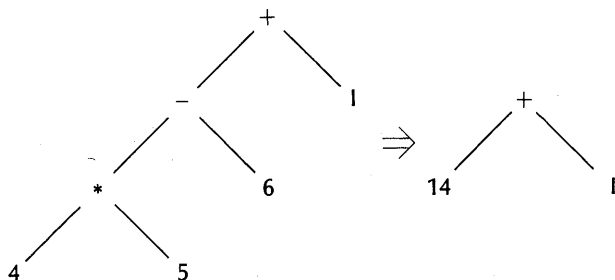


Figure 11.3. Associativity of $\{+, -\}$ applied to AST of figure 11.2.

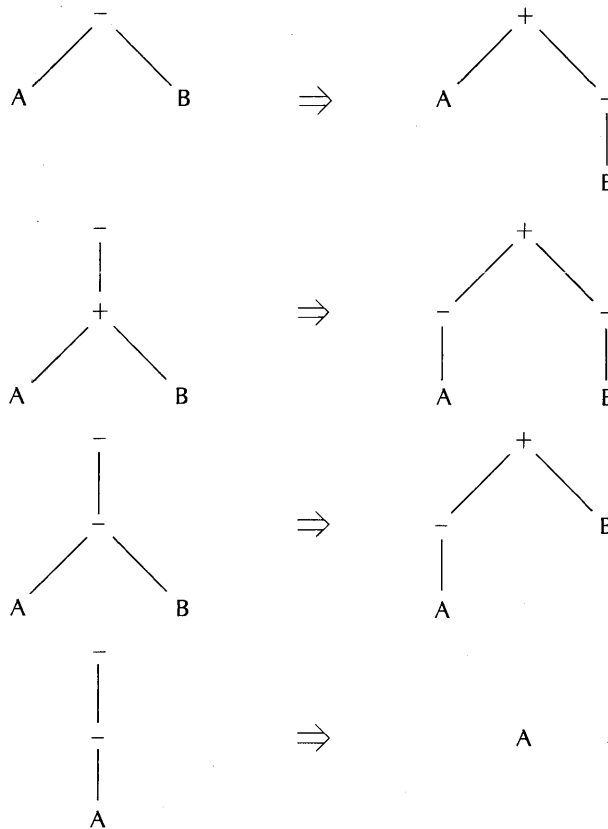


Figure 11.4. Transformations applied prior to flattening an AST for binary operations $\{+, -\}$.

single $+$ node. A node representing a parenthesized expression must therefore carry a mark that prevents its merger into a cluster.

Flattening is also useful for single-register machine coding. In such a machine, a binary arithmetic operation applies to an accumulator register A and some memory location M ; the result is left in A . Initially, A must be loaded from memory. Some expressions require temporary memory cells $T_1, T_2, \dots$. Thus the replacement statement

$$W := R + S + U * V;$$

would be coded without optimization as:

```

LOAD  R; {value of R to accumulator}
ADD   S; {add S to accumulator}
STOR  T1; {R+S must be saved to do
           the multiplication}

```

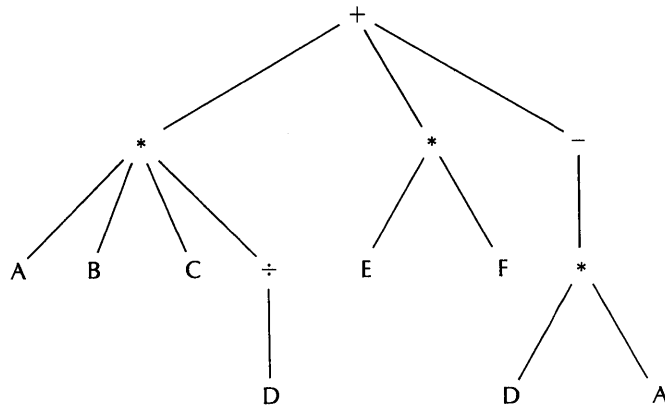


Figure 11.5. A flattened AST for the expression $A*B*C/D + E*F - D*A$.

```
LOAD  U;  
MPY   V;  
STOR  T2;  
LOAD  T1;  
ADD   T2;  
STOR  W; {9 instructions, 2 temporaries needed}
```

The flattened AST for this expression is shown in figure 11.6. For optimal single-register code, the * subtree should be the leftmost sibling of the + node, as indicated in figure 11.7. Then the single-register code for the tree of figure 11.7 is:

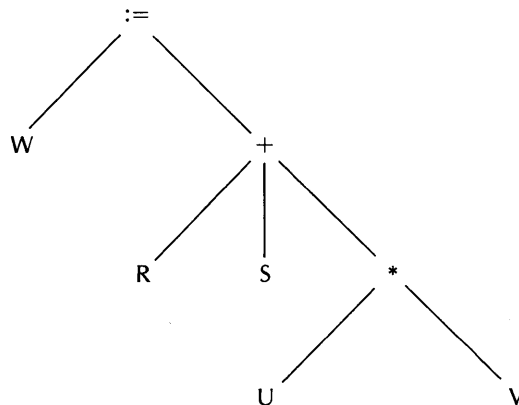


Figure 11.6. Flattened AST for statement $W := R + S + U*V$.

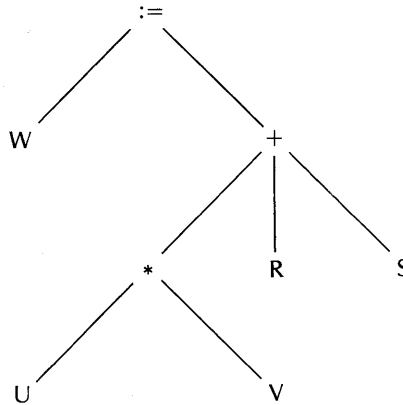


Figure 11.7. AST of figure 11.6 rearranged for optimal single-register code.

```

LOAD  U;
MPY   V;
ADD   R;
ADD   S;
STOR  W; {5 instructions, no temporaries}

```

A reasonable optimization rule is therefore: (1) transform binary subtraction and binary floating-point division nodes into binary addition and multiplication nodes, in order to produce as many clusters as possible, (2) merge the clusters, (3) arrange constant siblings of some cluster node to the right and complicated trees to the left. (We shall develop a more complete optimal algorithm later.)

Exercises

1. Design an algorithm that identifies clusters in an arithmetic AST and flattens them. Assume floating-point arithmetic only.
2. Discuss the possibilities of optimization of each of the following operations:
 - (a) $A \uparrow B$ {exponentiation}
 - (b) if L then A else B {selection of A or B, depending on L}
 - (c) case I of N1: A; N2: B; N3: C; . . . Ni: Z; end; {multi-way selection of an expression}

3. Design an algorithm that identifies and reduces an AST containing identity and null operations, e.g.,

$$0 * X, 0 + X, 1 * X, X / 1, X - X, \text{ etc.}$$

4. Show that commutativity and associativity can yield no improvement in the number of instructions for a single expression tree, on a pure stack machine with an unlimited number of registers. A *pure* stack machine has an unlimited stack and the instruction set:

```

LOAD n  {C(n) → C(tos); tos = address of stack top}
STOR n  {C(tos) → C(n)}
ADD     {C(tos - 1) + C(tos) → C(tos - 1); tos := tos - 1}
SUB     {C(tos - 1) - C(tos) → C(tos - 1); tos := tos - 1}
MPY     {C(tos - 1) * C(tos) → C(tos - 1); tos := tos - 1}
DIV     {C(tos - 1) / C(tos) → C(tos - 1); tos := tos - 1}

```

5. Suppose that, in addition to the instructions given in exercise 4, a stack machine has some memory reference instructions that operate between a memory location and the stack top. Would it be possible to optimize evaluation of an expression tree using these memory reference instructions? If so, give some examples. The additional instructions would have the form:

```

ADDM n {tos + C(n) → tos}
SUBM n {tos - C(n) → tos}
MPYM n {tos * C(n) → tos}
DIVM n {tos / C(n) → tos}

```

11.3. Optimal AST Evaluation for a Multiregister Machine

The reported work in code optimization can be divided into two general classes: the class of *code-improvement methods* and the class of *provably optimal methods*. The latter class is much smaller than the former, undoubtedly because it is relatively easy to develop a heuristic method that, when it applies, improves the efficiency of the emitted code; it is much harder to prove that some general algorithm yields a code sequence that is absolutely minimal. Indeed, several important optimization questions are either undecidable or can only be solved by an algorithm of exponential complexity, a so-called NP-complete algorithm [Aho 1974b]. In the face of undecidability

or an NP-complete problem, heuristic code-improvement represents the only practical approach to an optimization.

One of the few optimal code generation algorithms is from Meyers [1965] and Nakata [1967], as expressed and analyzed by Sethi and Ullman (Sethi [1970]). This algorithm generates optimal code from a binary expression tree, for a multiregister machine. Three variations on the basic algorithm are developed, one that assumes that no operation is commutative or associative, a second that assumes that some operations are commutative, and a third that assumes that some are both commutative and associative.

It is important to note that Sethi's work does not address the problem of expression tree optimization, where the tree contains common subexpressions or of a set of expression trees that might be partially merged. These questions will be examined later.

11.3.1. The Machine

The commands permitted by the Meyers-Nakata-Sethi algorithm are of four types, for a machine with an unlimited amount of general memory and $NR \geq 1$ general purpose registers:

LOAD M,R: Copy contents of memory cell M to register R.

STOR M,R: Copy contents of register R to memory cell M.

OP R1,M,R2: Perform a binary operation OP (non-commutative in general) on register R1 and memory M, the result to replace contents of register R2. Neither R1 nor M is changed by the operation, unless $R1 = R2$.

OP R1,R2,R3: Perform a binary operation on registers R1 and R2, the result to replace contents of register R3. Any two or three of the registers may be the same. The contents of R1 and R2 are unchanged, unless one or both is register R3.

Note that a binary operation OP R1,M,R2 does not have a counterpart OP M,R1,R2. For example, the machine may have a subtraction operation $R - M$, but no "reverse" subtraction $M - R$. This restriction exists in many commercially available computers, and is therefore a realistic one to consider. If both operations exist, then every operator could be treated as though it were commutative.

Let us assume that the tree's value will be left in a register. Then some trees will require no STORs; there will be sufficient registers to hold all intermediate values. However, no matter how large NR is, there are trees that will require some STORs of intermediate results. The problem is to identify those leaf nodes that must be LOADED, those that must be stored, and the order of performing all the operations. Every operator node in the tree will

require one OP instruction regardless of the manner of loading operands and storing results, so that a minimal solution is one with the fewest LOAD and STOR instructions.

Considering the number of degrees of freedom possible, it is remarkable that an algorithm that finds the minimal code sequence in a reasonable time exists. We shall see later that if common subexpression optimization is also desired, then no algorithm that finds a minimal code sequence in a reasonable time exists—any such algorithm will require an exponential time complexity to find an optimal solution.

11.3.2. Tree Labeling

We begin with an algorithm that decorates each node N of the expression tree with the integer quantity $\text{LABEL}(N)$. This quantity will represent the number of registers needed to evaluate the subtree rooted in N without any STOR instructions. The algorithm invokes several tree functions explained in the comments.

```

procedure TREELABEL( $N$ : integer): integer;
  {a recursive procedure that accepts a
   node  $N$  and returns a label for the node}
begin
  if LEAF( $N$ ) then
    {LEAF( $N$ ) is true if  $N$  has no
     children}
    if  $N \neq \text{ROOT}$  then {ROOT = AST tree root}
      return(if  $N = \text{LEFTCHILD}(\text{PARENT}(N))$  then 1 else 0)
      {PARENT( $N$ ) is the parent of node  $N$ ;
       LEFTCHILD( $N$ ) is the left child of  $N$ }
    else return(1) {tree consists of one node}
  else
    begin { $N$  has children}
      var L1, L2: integer; {two temporary labels}

      L1:=TREELABEL(LEFTCHILD( $N$ ));
      L2:=TREELABEL(RIGHTCHILD( $N$ ));
      {get the labels of the child subtrees}
      return(if L1=L2 then L1+1 else
              MAX(L1,L2)) {maximum of the two labels}
    end
  end {of procedure}

```

TREELABEL is a bottom-up algorithm; the label for some node can be determined only if it is a leaf node or if the labels for its children are known.

Suppose that a node N is a leaf node. If a left leaf, then we need one register to hold its value in order to perform its parent operation. If a right leaf, its parent operation can be performed from memory, requiring no register. Of course, the left subtree of an operation with a right child leaf will require at least one register.

Now suppose that node N is an internal node. Let its child subtrees carry labels $L1$ (left) and $L2$ (right). If $L1 = L2$, then an equal number of registers are required to evaluate each of the subtrees; note that in this case $L1$ and $L2$ must be at least 1. We need one more register to carry out the operation of node N , to hold the value of the left subtree while the right subtree is being evaluated, or vice versa, hence the label for N is $L1 + 1$. If $L1 \neq L2$, then at least one register is available for one of the subtrees (the one with the lesser label); no additional registers are needed to evaluate node N , hence $L(N) = \text{MAX}(L1, L2)$.

11.3.3. Optimal code generation

We now introduce an optimal code generation algorithm, procedure EVALUATE. This is a top-down algorithm that operates on an expression tree decorated with the labels developed in the TREELABEL procedure above. It requires some more functions as follows:

EMITLOAD(M, R): Emits a LOAD instruction, LOAD M, R .

EMITSTOR(M, R): Emits a STOR instruction, STOR M, R .

EMITOP($OP, R1, S, R2$): Emits a binary operation $OP\ R1, S, R2$; OP is the operation, $R1$ and $R2$ are registers, and S is a or a memory location.

OP(N): This is the operator associated with node N . (N must be an internal node.)

LABEL(N): The label associated with node N , an integer, as determined by TREELABEL.

MEMLOC(N): The memory location associated with node N ; N must be a leaf node. We assume that every leaf node is associated with a memory location; no variable is already in a register.

ALLOCATE: A typed procedure that returns a temporary memory location. Temporary locations are needed when the register set cannot hold all the intermediate tree values.

RELEASE(T): Returns a temporary location T to a pool. Those in the pool may subsequently be used for other intermediate results.

Now the code emitting algorithm is as follows. The comments {A}, {B} indicate sections of the algorithm that will be expanded upon in the next section.

```

procedure EVALUATE(N, M: integer);
  {N is a subtree root node.
   M is the first of a set of registers
   available for the evaluation of the
   subtree rooted in N; the available
   registers are M, M+1, . . . ,
   where the machine contains NR
   registers. Initially, EVALUATE is
   called with N=ROOT and M=1. The
   result of an EVALUATE call is the
   value of the subtree N left in
   register N}
begin
  if LABEL(N)=1 then
    begin
      if LEAF(N) then
        begin {N must be a left child}
          {A}  EMITLOAD(MEMLOC(N), M)
              {a left leaf must be LOADED}
        end
      else
        begin {the right child must be a leaf}
          {B}  EVALUATE(LEFTCHILD(N), M);
              EMITOP(OP(N), M,
                  MEMLOC(RIGHTCHILD(N)), M)
        end
      end
    end
  else
    begin {LABEL(N) > 1 here;
          N must have children}
      var N1, N2, L1, L2: integer;

      N1:=LEFTCHILD(N);
      N2:=RIGHTCHILD(N);
      L1:=LABEL(N1);
      L2:=LABEL(N2);

      if MIN(L1,L2) ≥ NR then
        begin
          var T: integer;

```

```

{C}    EVALUATE(N2, M); {evaluate right child}
        T:=ALLOCATE; {allocate a cell}
        EMITSTOR(T,M); {register M put away}
        EVALUATE(N1,M);
        EMITOP(OP(N),M,T,M);
        RELEASE(T) {T no longer needed}
    end
    else
        if L1  $\neq$  L2 then
{D}    begin {Note: MIN(L1,L2) < NR here}
        if L1 > L2 then
            N1 := N2; {exchange N1, N2; now
                        N1 is the node with the least label}
        if MIN(L1,L2) > 0 then
            begin
                EVALUATE(N2,M);
                EVALUATE(N1,M+1);
                if L1<L2 then
                    EMITOP(OP(N),M+1,M,M)
                else EMITOP(OP(N),M,M+1,M);
            end
        end
        else
            begin {lesser of L1, L2 is 0}
                EVALUATE(LEFTCHILD(N),M);
                EMITOP(OP(N), M, RIGHTCHILD(N),M);
                {N's right child must be a leaf}
            end
        end
    end
    else
{E}    begin {L1 = L2 and L1 < NR}
        EVALUATE(N1, M);
        EVALUATE(N2, M+1);
        EMITOP(OP(N),M,M+1,M)
    end
end {of procedure}

```

11.3.4. Discussion

Section {A} deals with the case of a leaf node N with label 1. N must be a left child of its parent. No operation has its left operand in memory, so we must load the value of N. We use the next available register M for this purpose.

In section {B}, the right child of N must be a leaf; the label of N cannot be 1 otherwise. We can evaluate the left child of N with registers $M, M+1, \dots, NR$ available, and the operation of node N can then be done with no additional registers.

In section {C}, both children of N require more registers than are available. We must therefore allocate a temporary cell T and use it to save register M . We then evaluate the right subtree and save the result. When the left subtree is evaluated, its result will be in register M , ready for the operation $OP(N)$. After the operation, the temporary cell T is no longer required. Note that in the Pascal procedure, T is a local variable and is therefore preserved in the recursive calls.

In section {D}, one of the labels (L_1, L_2) is less than NR . This means that at least one register is available for the operation $OP(N)$. A special case arises if the lesser label is zero; it must be the label of the right child, a leaf node. We can therefore evaluate the left subtree with registers $M, M+1, \dots$ available and then operate directly on the register M and the right child leaf.

If the lesser label is not zero in section {D}, we evaluate that subtree with the lesser label first, then the subtree with the greater label. The former result is left in M and the latter in $M+1$. Finally, the operation $OP(N)$ is a register-register operation, with the result in register M .

In section {E}, the subtree labels are equal and less than NR . We therefore have an available register for the operation, which will be a register-register operation.

Example. Consider the expression tree of figure 11.8. The nodes are named

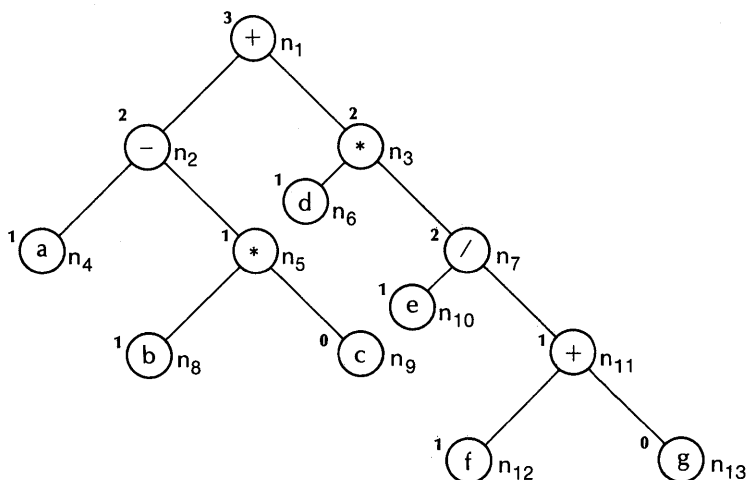


Figure 11.8. An example expression tree labelled with the minimum number of registers required to evaluate each subtree.

$n_1, n_2, \dots, n_{13}$, and carry labels ranging from 0 through 3. The leaf nodes carry operands $a, b, c, \dots, g$. The tree represents the expression $(a - b * c) + (d * e / (f + g))$.

For example, the root node carries label 3 since its children each carry label 2. Node n_3 carries label 2 since its children carry labels 1 and 2 respectively.

We may trace the recursive calls of EVALUATE as follows, for a machine with $NR = 2$ registers, $R1$ and $R2$:

EVALUATE($n_1, 1$) yields the calls and code:

```
EVALUATE( $n_3, 1$ );
STOR  $T_1, R1$ ;
EVALUATE( $n_2, 1$ );
ADD  $R1, T_1, R1$ ;
```

EVALUATE($n_3, 1$) yields the calls and code:

```
EVALUATE( $n_7, 1$ );
EVALUATE( $n_6, 2$ );
MPY  $R2, R1, R1$ ;
```

etc. (See Exercise 1)

When all the EVALUATE calls are traced, the following code sequence is found:

```
LOAD  $e, R1$ ;    {load operand  $e$ }
LOAD  $f, R2$ ;
ADD  $R2, g, R2$ ;    {e.g.,  $C(R2) + g \rightarrow C(R2)$ }
DIV  $R1, R2, R1$ ;
LOAD  $d, R2$ ;
MPY  $R2, R1, R1$ ;
STOR  $T_1, R1$ ;
LOAD  $a, R1$ ;
LOAD  $b, R2$ ;
MPY  $R2, c, R2$ ;
SUB  $R1, R2, R1$ ;
ADD  $R1, T_1, R1$ ; {result left in register  $R1$ }
```

Since the tree carries a maximum label 3, at least one temporary location is needed in any code sequence.

Exercises

1. Complete the list of EVALUATE calls and verify the code sequence given above.

2. Develop optimal code sequences for machines with $NR = 1$ and $NR = 3$, for the AST of figure 11.8.
3. Prove that EVALUATE evaluates a tree with no stores when as many registers as the label of the root node are available. (*Hint*: Is the label of the root node the largest of any of the tree labels?)
4. A *minor node* is a leaf node that is the left child of its parent. Show that the number of LOAD's in an evaluation program must be at least equal to the number of minor nodes in the expression tree.
5. A *major node* is an internal node, both of whose children have LABEL values greater than or equal to NR , the number of registers in the machine. Show that the number of STORs in an evaluation program is at least equal to the number of major nodes in the tree.
6. Prove that EVALUATE generates an optimal code sequence. *Hint*: Use examples 4 and 5 above, and prove the following two lemmas:
 - Lemma 1. The number of stores generated by EVALUATE is equal to the number of major nodes in the tree.
 - Lemma 2. The number of loads generated by EVALUATE is equal to the number of minor nodes in the tree.

11.3.5. Commutative Operators

When a commutative operator is present in an expression tree, we have an opportunity to interchange its children with the objective of reducing the number of loads or stores. Considering Lemmas 1 and 2 in exercise 6 above, we see that the number of stores is unaffected by such an interchange. However, the number of loads can be reduced, if the number of minor nodes is thereby reduced. Clearly, those commutative operators that carry a leaf left child and a nonleaf right child are candidates for child interchange, and the interchange will reduce the number of loads. We therefore have the rule:

Commutative Operator Reduction Rule. For every node N such that: (1) $OP(N)$ is commutative, and (2) $LEAF(LEFTCHILD(N))$, and (3) $\sim LEAF(RIGHTCHILD(N))$, do: interchange ($LEFTCHILD(N)$, $RIGHTCHILD(N)$).

The resulting tree will yield an optimal code sequence under algorithm EVALUATE.

For example, the tree of figure 11.8 is transformed into the tree of figure 11.9 through an interchange of the subtrees rooted in nodes n_6 and n_7 . We thereby reduce the code sequence by one LOAD.

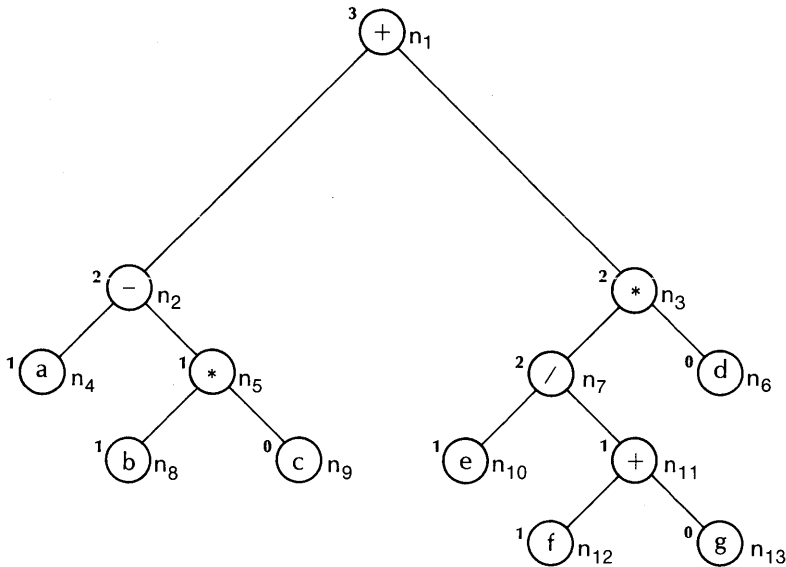


Figure 11.9. Commutative optimization of tree of figure 11.8.

11.3.6. Associative and Commutative Operators

Associativity by itself is of little value in reducing the number of loads and stores in a code sequence; however, the combination of associativity and commutativity (A–C) is of value.

We use the notion of a cluster developed earlier. A *cluster* is a set of A–C nodes associated with the same operator that forms a maximal tree. When some clusters exist, the tree may be reorganized in any fashion by rearranging the subtrees of any cluster.

The labeling rules for an associative tree are slightly different than those expressed in TREELABEL. The children of every node are first arranged in decreasing label sequence from left to right (the left most child has the largest label). Let L_1 and L_2 be the labels of the two left most children of some node N . Then $\text{LABEL}(N) = L_1$ if $L_1 > L_2$, and $L_1 + 1$ if $L_1 = L_2$.

Given such a labeled associative tree and any cluster node N in such a tree, e.g., figure 11.10(a), we then transform the cluster subtree into the binary subtree shown in figure 11.10(b), with root N' . This transformation is made on every associative cluster node. The resulting tree then yields a minimal code sequence under the EVALUATE algorithm.

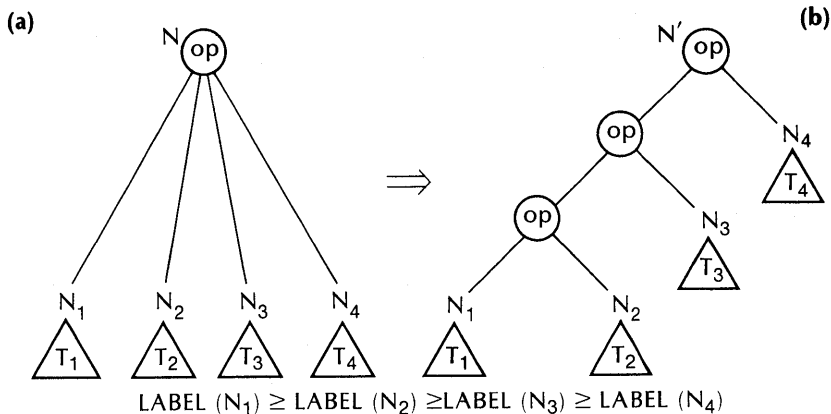


Figure 11.10. Transformation of an associative tree into an optimal binary tree.

Exercises

1. Construct a tree for the expression

$$a * (b + (c * d) + e) * f + g + h * j$$

and apply the associative-commutative transformations described above. Generate the minimal code sequence for the expression.

2. Show that the associative-commutative transformation yields a minimum code sequence.

11.4. Code Improvement over a Sequence of Statements

The generation of optimal code for a single expression is hardly worth while in an average program. Knuth [1971], in a study of a large sample of FORTRAN programs, found that of all the assignment statements in his sample, 68% were a simple replacement of the form $A = B$, with no arithmetic operators and 13% were of the form $A = A \text{ op } B$, with the first operand on the right the same as the replacement variable. Hence only 19% have a more complex structure, most of which apparently involve very few operators.

Of the variables, 58% were not indexed, 30.5% had a single index, 9.5% had 2 and only 1% had 3 indices (the maximum number under the Fortran in use at Stanford and Lockheed, the sources of the test programs).

An optimization for expression trees only will probably have little effect on the efficiency of a typical program. Usually few statements will have an

expression tree large enough for the algorithm of the preceding section to yield an improvement. Of course, these expressions might reside in a frequently executed section of the program, magnifying their importance to the program's performance.

The prospect of eliminating common subexpressions and register assignment over a sequence of assignment statements, called a *block*, is more promising.

Knuth also studied the improvement that would result from different levels of optimization. At the lowest level, blocks were considered for constant subsumption, rearrangement and redundancy elimination. A reasonable register allocation strategy was also employed. At a second level, the flow of control among blocks was also considered to achieve global constant subsumption and rearrangement and some strength and frequency reduction. Improvements were found at each level, the second yielding somewhat greater relative improvement (40% increase and 170% increase over the raw non-optimized code, respectively). A further improvement could be made by exploiting idioms for the IBM System/360 system, other than its basic multiregister organization.

Code can be improved in a sequence of statements in several ways: a larger field of expressions is available for identifying common subexpressions; register allocation (for a multiregister machine, at least) can be improved; and loop-invariant expressions can be moved outside a loop.

None of these is possible without the development of methods for determining the status of all the program variables at any one point. For example, the value of a variable may be undefined at certain locations in the program, then defined later. A given definition will hold through some sequence of statements, then be replaced by another definition. Finally, a variable may become *dead* after some reference, i.e., it is no longer used after the reference. The status of the variables obviously influences register allocation and the identification of common subexpressions and of loop invariants.

We see that development is needed along three lines. We need: (1) a more general optimization plan for a block, that can identify common subexpressions and allocate registers efficiently, (2) an analysis of program control flow and its effect on the status of each variable, and (3) the exploitation of variable status in intra-block as well as inter-block optimization.

11.4.1. Blocks

A block consists of a sequence of assignment statements $S_1, S_2, \dots, S_b$ that are executed in that order, and such that control can only pass into the first statement, from any one statement to its successor, and out of the last statement. No program branch may enter the block except to its first statement.

Each statement is a simple assignment of the form

$$C \leftarrow \theta B_1 B_2 \dots B_n$$

where θ is an n -ary operator, $B_1, B_2, \dots, B_n$ are its operands, and C receives the single scalar result of the operation. C is said to be *defined* by the statement, and the B_i are said to be *referenced* or *used*.

This schema is sufficiently general to cover many common program statements. A simple replacement statement of the form

$$X := A + B * C$$

can be subdivided into a sequence of simple assignments by introducing a temporary variable T :

$$T \leftarrow *, B, C$$

$$X \leftarrow +, A, T$$

A function call returns a single value; however, if some of its parameters are called by reference or by name, then the possibility exists that they have been defined. Such parameters may be marked *defined* by the null operation DEF:

$$B \leftarrow \text{DEF}$$

indicates that B has been defined in some previous statement, such as a function call:

$$R \leftarrow \text{FN}, P_1, P_2, \dots, B, \dots, P_r$$

An indexed replacement, e.g.,

$$B(X) := E$$

involves an assignment for X and then for B ; however, since the compiler cannot determine which of the vector of B variables has been defined, it can only assume that any one of them has been. An array variable is therefore referenced or defined as a unit; the optimization algorithms cannot usually distinguish the members of the vector.

A vector move statement is similarly treated. Thus in PL/I, if A and B are compatible structures, the statement

$$A = B$$

represents an assignment to A of each of the components of B , and the optimization algorithms must deal with structures A and B as units.

11.4.2. Variables and Their Domains

A *variable* is some value created at run-time by a definition that is

preserved until another definition or the end of the program is reached. A variable is associated with some name, but a given name may represent several different variables.

Thus in the following statements, one variable associated with X is marked by the arrows ($\uparrow$):

```

X := X + Y;
↑
A := X + 5;
    ↑
X := X - Y;
    ↑

```

Note that the first reference and the last definition of X are not part of this variable; they are part of another variable, one that happens to carry the same name in these statements.

The *domain* of a variable is the set of all statements in the program that includes the definition statement, every reference statement, and every statement S such that a control path may pass from the definition through S to some statement containing a variable reference. The variable is said to be *live* at some statement if the statement is within its domain.

If the domain of a variable extends through the end of a block B, the variable is called an *output* variable of B. If the domain includes any statement passed through before reaching the beginning of the block, the variable is an *input* variable of B. All other variables are *local* or *temporary* variables; their domains are contained within the block.

The input variables of a block can be identified—they have some reference prior to a definition. Unfortunately, the output and local variables cannot be distinguished from an examination of the block alone; it is necessary to examine the control paths that lead from the block. Certain local variables can be identified: (1) the declared local variables, for a source language that permits variables to be declared at the head of any block, and (2) temporaries introduced through the reduction of arithmetic expressions to simple assignment statements.

11.4.3. Equivalent and Normal Blocks

Two blocks are said to be *equivalent* if they carry the same sets of input and output variables, and if, for every set of values of input variables, the resulting output variable values are the same upon execution of the block statements. One of the blocks may always be transformed into the other through a sequence of four equivalence transformations, as Aho [1974b] has shown. The four transformations are

1. Removal of useless statements and variables.
2. Identification of two computations producing the same value (common subexpressions).
3. Renaming of variables, e.g., temporary variables.
4. Interchange of two adjacent statements, under conditions that ensure preservation of equivalence.

A *normal* block is such that every variable carries a unique name. Any block can be transformed into an equivalent normal block by renaming certain of the temporary or input variables. For example, consider the block

```

X:=A+B;
Y:=A-B;
X:=X*Y;   {second use of variable name X}
Y:=Y*X;   {second use of variable name Y}

```

The names X and Y are associated with more than one variable each in this block; we therefore introduce new names X' and Y', to yield the following equivalent normal block:

```

X:=A+B;
Y:=A-B;
X':=X*Y;   {beginning of second variable X'}
Y':=Y*X';  {beginning of second variable Y'}

```

An input variable X must be renamed if a definition of X appears in the block. An input variable that is also an output variable need not be renamed. We adopt the convention that output variables will never be renamed.

Only normal blocks will be considered in the following sections.

11.4.4. Representation of a Block as a DAG

A normal block may be represented as a directed acyclic graph, or DAG, in much the same way that an expression can be represented as a tree. A DAG consists of a number of nodes connected by directed edges, such that no directed path is closed. A DAG will have one or more root nodes, with no indirected edges, and one or more leaves, with no outdirected edges. A node with at least one outdirected edge is an *internal* node. (The root nodes may be internal nodes). We will be interested only in DAGs for which every root is not a leaf.

Each node of a block DAG will be associated with a variable. In addition, each internal node will be associated with an operation.

A DAG is constructed from a normal block as follows:

1. The statements $S_1, S_2, \dots, S_b$ of a block will be considered in that order. Each statement will in general add nodes and edges to the block DAG.
2. Given a statement

$$C \leftarrow \theta B_1 B_2 \dots B_n$$

we add a new leaf node for every variable B_i not already in the DAG. A new node N_C for variable C and operation θ is added to the DAG. Then edges from node N_C to each node $B_1, B_2, \dots, B_n$ are added. For example, consider the block

```
T ← +, A, B
C ← *, T, A
A ← /, C, A
F ← +, E, E
B ← /, T, D
```

where $\{A, B, D, E\}$ are input variables and $\{A, B, F\}$ are output variables. Since A and B are redefined in the block, we must rename the corresponding input variables. The equivalent normal block is

```
T ← +, A', B'
C ← *, T, A'
A ← /, C, A'
F ← +, E, E
B ← /, T, D
```

This block has the DAG shown in figure 11.11.

11.4.5. Value of a DAG

The *value of a DAG* is a set of values associated with its nodes. These values are determined by the values associated with the input variables and a DAG evaluation rule. Each node of a DAG is associated with a value, computed by the following rule:

DAG Evaluation Rule. If the node is a leaf, it must be an input variable; its value is therefore the value of that variable. If the node is internal, its value is the result of its operator applied to its children's values.

An input node's value clearly cannot be determined until each of its

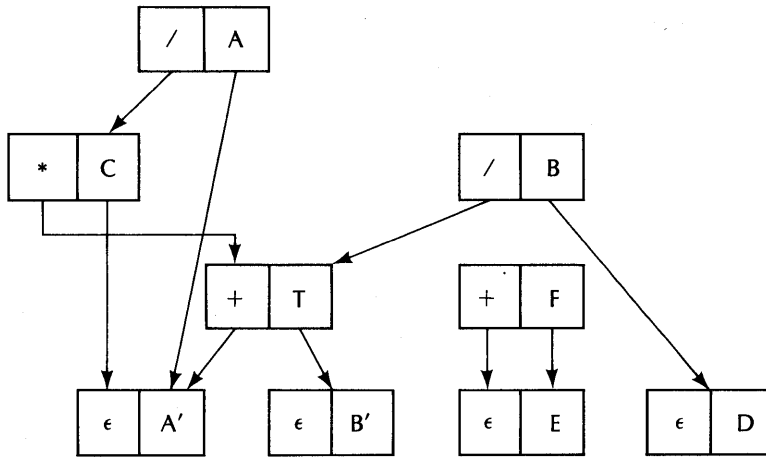


Figure 11.11. A directed acyclic graph (DAG) for the block $\{T \leftarrow A' + B', C \leftarrow T * A', A \leftarrow C / A', F \leftarrow E + E, B \leftarrow T / D\}$.

children's values are determined. Now each node represents a unique variable because of block normality. Evaluation of some node results in fixing the value of its associated variable; however, no other node evaluation can affect that variable's value, hence every evaluation is uniquely determined, regardless of the order in which evaluation occurs. The DAG value is also clearly the value returned by the block.

We are therefore free to explore DAG evaluations without regard to the order in which the evaluations take place. Evaluation is subject only to the constraint that a node can be evaluated only if all its children have been previously evaluated.

Furthermore, all of the internal non-output nodes represent temporary variables. These may be assigned to registers and never be allocated a temporary memory location, for the sake of efficiency, even though the source program may have assigned a name to the variable.

11.4.6. Common Subexpression Identification

During construction of a DAG, the appearance of two subexpressions, such that the second one need not be recomputed, is easily detected by a minor change in the construction algorithm.

We first suppose that none of the block operators are commutative or associative. (This restriction will be lifted later.) We are interested in identifying a DAG node that represents a value that can be saved for use later in the block. Such a node represents a subexpression that appears twice in the block and that, in addition, will return the same value on its second appearance.

We are therefore interested in a pair of assignment statements

$$\begin{array}{ll} S_i: & C \leftarrow \theta B_1 B_2 \dots B_n \\ S_k: & C' \leftarrow \theta B_1 B_2 \dots B_n \end{array}$$

such that none of the B_m ($1 \leq m \leq n$) are defined in any statement S_j where $i < j \leq k$. That is, statement S_k will return the same value to C' as statement S_i returned to C only if none of the referenced variables have been redefined in the interim.

Now block normality guarantees that none of the B_i are redefined; if one were, we then have a reference of that variable prior to its definition and its name could not appear in both S_i and S_k . The mere existence of the pair of statements $\{S_i, S_k\}$ is sufficient to guarantee that C' will receive the same value as C . In terms of the DAG being constructed, when statement S_k is under consideration, each of the nodes $B_1, B_2, \dots, B_n$ will already be in the DAG, and a one-level tree with a root associated with operator θ and variable C will exist. We need only look for this situation, and add variable C' to the node that contains C .

We see that a node will carry more than one variable in this extension in general.

11.4.7. Use of Associativity and Commutativity

Suppose that some of the operators are commutative and still others are associative-commutative. We may identify common subexpressions that are equivalent under these arithmetically invariant transformations by two simple operations.

Before the DAG is built, the operands of every commutative operator are arranged in some canonical order. For example, they may be arranged in alphabetical order. Then when the DAG is built, a pair of commutatively equivalent nodes will be identified as common subexpressions.

After the DAG is constructed, we search for associative clusters of nodes. Two nodes P and Q belong to the same associative cluster if and only if: (1) an edge from P to Q exists, (2) they carry the same associative operator, (3) excluding P , Q has no ancestors, and (4) at most one of them carries an output variable. If P and Q belong to the same cluster, they may be merged into a single associative node with several children; the node's children may then be rearranged in any order conducive to DAG or code reduction (e.g., constant subsumption or register allocation.)

Should an associative transformation be applied during or after the DAG construction? We don't know. If it is applied during construction, then an expression $C + B + A$ is merged into a cluster; this merger will then prevent recognition of $A + B$, $B + C$, etc., later. If it is applied after construction, then we need to search the DAG for equivalent, or partially equivalent clusters,

e.g., $B + C + A$ will be one cluster and $C + A + B$ will be another one; the two could be merged, but will require some searching. On the whole, there is no evidence that an associative transformation in a DAG is worthwhile as a means of detecting common subexpressions. It is certainly worthwhile for the subsumption of constants and for improving register allocation.

11.4.8. DAG Reduction

All of the temporary variables may be eliminated from the DAG, once built, which will incidentally also remove any redundant operations. A node may still carry more than one output variable. If it does, we have a situation in which a single variable (i.e., one definitional value) is represented by more than one name. A common name can clearly be assigned to these variables.

Finally, any root node that does not contain an output variable name may be eliminated. Such a node represents a definition of a temporary variable that is never referenced.

Example

Consider the block

```
V ← A*B
T ← A+C
C ← B*D
X ← V+C
Z ← T-C
C ← V+C
```

with the input set $\{A, B, C, D\}$ and output set $\{C, V, Z\}$.

An equivalent normal block is

```
V ← A*B
T ← A+C''
C' ← B*D
X ← V+C'
Z ← T-C'
C ← V+C'
```

with the input set $\{A, B, C', D\}$ and the output set $\{C, V, Z\}$.

The evaluation of $V + C'$ is clearly redundant; the redundancy is detected during the DAG construction. The final DAG is shown in figure 11.12. We also see that X is useless; it appears in a root node, but is a temporary variable; it is associated with node n_4 . The local variables T and C' are associated with nodes n_2 and n_3 , respectively.

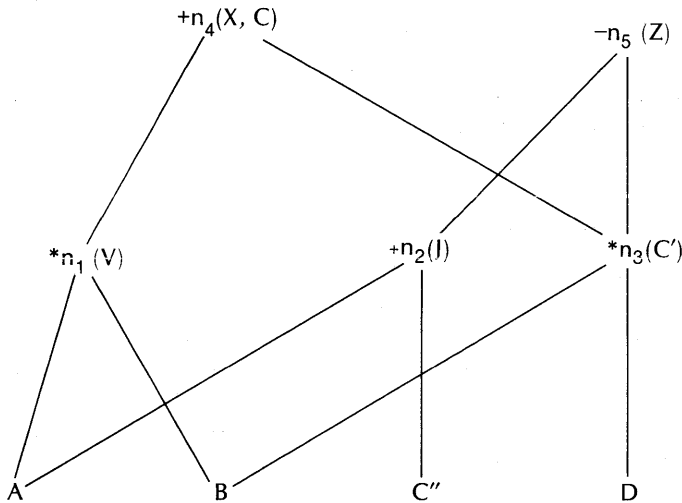


Figure 11.12. A DAG for the block $\{V \leftarrow A*B, T \leftarrow A+C'', C' \leftarrow B*D, X \leftarrow V+C', Z \leftarrow T-C', C \leftarrow V+C'\}$.

11.4.9. DAG Evaluation

Let us assume binary operations only in the following discussion, for the sake of simplicity. (This restriction is not hard to remove.) Also let us assume that a multiregister machine as defined in section 11.3 is to be used for evaluation of a DAG. This machine, it will be recalled, has LOAD and STOR operations between a memory location and a register, and binary operations between two registers or between a register and memory, the result going to a register.

We would like an algorithm that, given any DAG, will generate optimal code. Unfortunately, such an optimization is a large combinatorial problem—the computational complexity of any optimal algorithm increases exponentially with the number of nodes (Aho [1977]). Even simpler target machines, e.g., machines with one register or with an unlimited number of registers, have no optimal algorithm of less than exponential complexity.

However, there are several heuristic algorithms for single and multiregister machines; these algorithms are reviewed in Aho [1977]. Let us examine one of them in some detail, suggested by Waite [1974b], called the *top-down greedy* (TDG) algorithm.

The TDG algorithm builds a list L of internal nodes. This list, when used in reverse, will yield a reasonably good node evaluation sequence for a multiregister machine of the class described in section 11.3, containing NR registers.

```

procedure TDGP(N, K: integer);
  {N is a DAG node, K is the number of
   available registers}
  if (K ≤ 1) and (N is an internal node
    all of whose parents
    are on the list L) then
  begin
    Add (N) to list L;
    TDGP(LEFTCHILD(N), K-1)
    TDGP(RIGHTCHILD(N), K);
  end;

  {Now the main program}

  while (not all internal nodes are
    on the list L) do
    (select an internal node N, all of
     whose parents are on L, and
     perform TDGP(N, NR));

```

The TDG algorithm begins by selecting some root node of the DAG (no parents). Then TDGP will place it on the list L. TDGP will then pursue the left branches of its current node, adding them to the list until some node is found with a parent not on the list, or until no more registers are available. It then pursues the right branches in the same way.

The selection of a node could be influenced by some additional tests. Given a set of possible nodes (i.e., none are on the list and all have no ancestors), choose a node n whose left descendant n' is such that n is the only ancestor of n' not on the list. If there is more than one such n , look at the left descendants of n' , etc. This strategy will yield a sequence of DAG nodes that tend to be linked together; the number of temporary memory references will thereby be minimized to the extent possible under such a simple approach.

Example

Consider a machine with $NR=2$ (two registers) and the DAG of figure 11.13, with input variables A, B, C, D and output variables X and Y.

TDG yields the list

7, 5, 6, 3, 4, 1, 2

We then use this list in reverse, along with a reasonable register allocation scheme (one is described later), to obtain the code sequence given below. The following table indicates the node or variable currently in one of the two registers {R1, R2}.

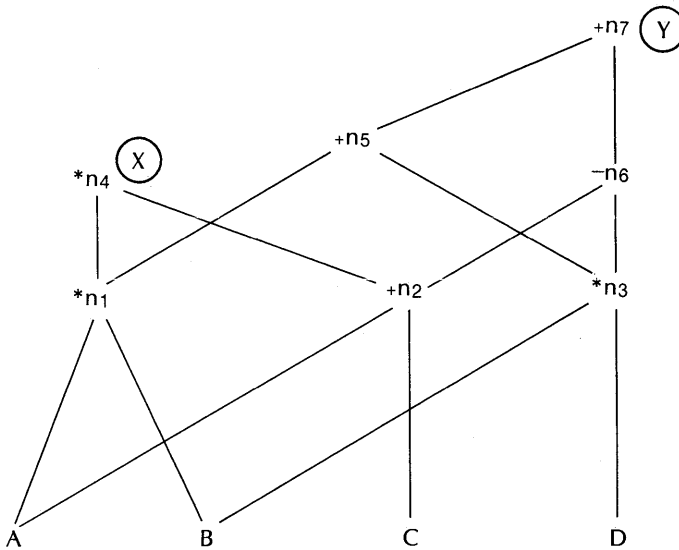


Figure 11.13. An example DAG (see text).

	contents of	
	R1	R2
R1 $\leftarrow$ A	A	
R2 $\leftarrow$ R1 + C		n_2
R1 $\leftarrow$ R1 * B	n_1	
T1 $\leftarrow$ R1 {temporary}	—	
R1 $\leftarrow$ R1 * R2	n_4	
X $\leftarrow$ R1		
R1 $\leftarrow$ B	B	
R1 $\leftarrow$ R1 * D	n_3	
R2 $\leftarrow$ R2 - R3		n_6
T6 $\leftarrow$ R2 {temporary}		—
R2 $\leftarrow$ T1		n_1
R1 $\leftarrow$ R2 + R1	n_5	
R1 $\leftarrow$ R1 + T6	n_7	
Y $\leftarrow$ R1		

Exercises

1. Construct a DAG for the block

```

X := A*(B+C);
Y := (B+C)/A;
X := X-A;
B := X/Y;
Y := A+B+C;

```

X and Y are the output variables; A, B, and C are input variables.

2. Show that a TDG list for the DAG of figure 11.12 is:

5, 2, 4, 1, 3

Develop a code sequence for the list of exercise 2, for $NR=2$. Can you find a shorter sequence for some other list?

4. Let the reverse of a TDG list be $n_1, n_2, \dots, n_r$, and let the nodes be evaluated in that order. Show that when any node N is to be evaluated, all of its children (if any) have been previously evaluated.

11.4.10. Register Assignment and Code Generation

Suppose that we are given an ordering of the internal nodes such as that provided by the reverse of the list generated by TDG. We may then assign registers and emit code by a heuristic algorithm similar to the following.

1. We maintain a list of assignments of registers to nodes. A register R is assigned to a node N if, at that point in the code sequence, node N's value is carried by register R. Initially, no register is assigned to any node. A register is said to be *available* if it is not assigned to a node. We also assign a level number to each node in the DAG; initially, this will be the number of parents of the node. As operations are coded, the level numbers will be reduced to 0.

2. Then the following procedure EMITNODE is executed for each node N on the node list generated by TDG or some other algorithm:

```

procedure EMITNODE(N);
  {N is a DAG node, all of whose
   children have been evaluated}
begin
  if (LEFTCHILD(N) value is held in register R) then
    begin
      if LEVEL(LEFTCHILD(N))=1 then
        begin {a level of 1 means that after this use, the
               left child's register will be available}

```

```

    EMITOP(OP(N),R,VALUE(RIGHTCHILD(N)),R);
    {the right child may or may not be in a register;
     either instruction form may be emitted}
    ASSIGN(R,N) {assign R to node N}
end
else
begin {left child will be needed after this operation}
    var b': integer; {declare a temporary}
    R' := ALLOCATE; {get a register; it could be R}
    EMITOP(OP(N),R,RIGHTCHILD(N),R');
    ASSIGN(R',N)
end
end
else
begin {left child is not in a register}
    var R: integer;
    R := ALLOCATE; {allocate a register}
    EMITLOAD(VALUE(LEFTCHILD(N)), R);
    {load leftchild in register R}
    ASSIGN(R,LEFTCHILD(N));
    EMITNODE(N); {try again; left child is now in
    a register}
    return {cleanup was done on previous call}
end;
{begin some cleanup for node N}
if (N carries an output variable X) then
    EMITSTOR(X, REGISTER(N)); {emit a store
    to X from the register assigned to N}
    DECRLEVEL(N) {adjust levels and release registers}
end;
procedure DECRLEVEL(N: integer);
begin
    {this decrements the level numbers of the
    children of node N. If the level of N or any of
    its children is then 0, the corresponding registers
    are released}
    var NLC, NRC: integer; {temporary node numbers}

    NLC:=LEFTCHILD(N);
    NRC:=RIGHTCHILD(N);
    LEVEL(NLC):=LEVEL(NLC)-1;
    LEVEL(NRC):=LEVEL(NRC)-1;
    if LEVEL(NLC)=0 then RELEASE(NLC);
    {if a register or temporary is not assigned to node N,

```

```

        RELEASE(N) does nothing; otherwise it makes
        the register or temporary assigned to N available}
    if LEVEL(NRC)=0 then RELEASE(NRC);
    if LEVEL(N)=0 then RELEASE(N)
end;

procedure ALLOCATE: integer;
begin {allocates a register, returning its number}
    if (any register R is available) then return(R)
    else
        begin
            var T: integer;
            T:=ALLOCATETEMP; {allocate temporary mem-
                             ory cell}
            if (any register R is assigned to a node N such
                that N is not a left descendant of any of
                its ancestors) then
                EMITSTOR(R, T) {save R in cell T}
            else
                begin {no reasonable basis for choice}
                    Choose any register R assigned to a node N;
                    EMITSTOR(R, T) {save R in cell T}
                end;
            ASSIGN(T,N); {temporary T assigned to node N}
            return(R)
        end
    end
end

```

These procedures exploit several possibilities that can arise. If a register is needed and one is available, then it is used. If a register is needed and none are available, then we look for one assigned to a node that is only a right descendant of its ancestors; such a node can later be accessed directly through its temporary memory cell and need not be allocated a register. Barring these possibilities, there is probably no good basis for a register choice.

The register allocator is responsible for saving the present contents of an assigned register; that value will be needed later, since the register cannot be assigned to a node at the end of an EMITNODE call if the level of the node has fallen to zero.

Exercises

1. Generate a code sequence for the DAGs of Figs. 11.12 and 11.13 using TDG and EMITNODE.
2. Show that EMITNODE emits correct code for a DAG, given a node list with the evaluation order property that a node is selected for evaluation only if each of its children has been evaluated.

3. Discuss extensions to TDG and EMITNODE for n-ary operators, e.g., a function call, a MAX operator, etc. Can an n-ary operator be transformed into a sequence of binary operators?
4. Devise data structures for a block, a DAG, and whatever else is needed to implement a code generation system. Write Pascal procedures that emit code for a given block, expressed as a sequence of binary assignment operations.
5. Arbitrary choices are possible in both ALLOCATE and TDG. Where? Suppose, as an optimizing strategy, that these choices are made through a backtracking, nondeterministic machine. We can then generate all possible equivalent code sequences and choose the shortest one. Show that such a system must halt in finite time. Is this computationally feasible? How would the number of operations vary with the size of the DAG? (A measure of the size of a DAG might be the number of nodes).

11.5. Data Flow Analysis

In considering optimization of blocks, we need information regarding the definition and use of data items that cannot be found by an examination of the block alone. For example, we cannot distinguish output and temporary data items; we need to know whether any reference to some data item will lie in a control path leading from the block, uninterrupted by another definition. In short, we cannot determine the domain of the block variables by information in the block alone.

If we had a complete data flow analysis of a program, we could provide, in addition to block optimization, some useful tests and other optimizations as follows:

1. A sneak path, along which a reference to some data item is not preceded by a definition, can be detected. The programmer should be made aware of such paths, although the logic of the program may be such that the path can never be completely followed to the reference point.
2. Redundant or useless code can be detected. A redundant assignment statement is one that is followed eventually by an identical one. A useless assignment is one for which no subsequent reference exists.
3. Redundant or useless variables can often be detected. A redundant variable is one for which another variable always carries the same value. A useless variable is one for which only definitions and no references exist. A redundant variable can only be detected under certain circumstances; in general, it is not possible to determine from a static analysis of a program whether two variables carry identical values.

4. Register allocations might be carried from one block into others, or through a block, if it appears that some code savings might result.
5. Loop invariant variables can often be identified.

Data flow analysis is based upon a *control flow graph*. Given a program, its flow graph G consists of a connected, directed graph (not necessarily acyclic) having a single entry node n_0 . Graph G consists of a set of nodes $N = \{n_1, n_2, \dots, n_m\}$, representing blocks of instructions, and a set of edges E , connecting pairs of nodes, that represent the branching paths between blocks.

Each block has one entry and one exit point and consists of the largest sequence of program steps that can be so formed. A block might also consist of the instructions of a set of blocks connected together in such a manner as to contain only one entry and one exit point; we call such a block an *extended block*.

The detailed nature of a branch decision is ignored in data flow analysis, as are the details of the calculations within a block. The only information of interest is (1) the possible branches, (2) the definitions of data items, and (3) the uses of data items. It is entirely possible that the branching decisions in a program are such that some paths can never be followed during execution; however, for our purposes, we must assume that any directed path can be followed during execution.

11.5.1. Definitions

A *definition* of a data item R is some statement that assigns a value to R , replacing a previous assignment. A *use* of R is some statement that requires the current value of R in a computation.

A *locally available definition* of a data item R in a block B_i is the last definition of R appearing in that block. We denote the set of available definitions for block B_i by DB_i .

Any definition of an item R is said to *kill* all definitions of R that can reach the block containing the definition. Any definition of a data item R that can reach a block are *preserved* by the block if R is not defined in the block. The set of all definitions in the program that are preserved in block B_i will be denoted PB_i .

A definition d in a block B_1 is said to *reach* block B_2 if both of these conditions hold:

1. Definition d is locally available from B_1 ,
2. Definition d is preserved on some directed path P from B_1 to B_2 (but not necessarily on all paths.) Path P may be null.

The notion of *reaching* is essentially that the value of a data item X assigned in block B_1 can somehow get through to B_2 , without an intervening redefinition of X . The set of definitions that reach a block B_i will be denoted R_i .

A definition in a block B is said to be *available* at block B' if it is locally available in B and can reach B'. We denote the set of available definitions at a block B_i by A_i.

A *locally exposed use* of a data item X in a block B is a use of X in B that is not preceded in the block by a definition of X. A variable with a locally exposed use in block B is an input variable of B.

A use of a data item X is *upwards exposed* in block B if it either is locally exposed in B, or if there exists a path through B and to some other block B' that does not contain a definition of X, and such that X is locally exposed in B'. Here the notion is that some use of X in block B' can be the terminus of some path running through a number of blocks, including a block B; that use is then upwards exposed at B. The set of upwards exposed uses of a block B_i will be expressed as a set U_i. Note that this set contains uses that appear in block B and elsewhere in the program.

A definition d is *active* or *live* at block B_i if d reaches B_i (i.e., $d \in R_i$) and there is an upwards exposed use at B_i of the data item defined by d. The set of live definitions at block B_i will be denoted L_i. Clearly,

$$L_i = R_i \cap U_i$$

The set intersection of R_i and U_i corresponds to the two requirements that there be an upwards exposed use and that definition d reach block B_i.

Examples.

Consider the following block:

```

K:=3;      {1}
J:=3*K;    {2}
N:=J-2;    {3}
J:=3*N;    {4}
N:=J+6;    {5}

```

The *locally available* definitions out of this block are those for J, K, and N, lines 1, 4, and 5. The definitions in lines 2 and 3 are *killed* by the subsequent lines 4 and 5.

There are no locally exposed uses for this block; these uses would be input variables, and there are none. If the first statement were

```
K:=J+N;
```

instead, then these uses of J and N would be locally exposed.

Now consider the following block:

```

K:=17;     {1}
J:=M+3;    {2}
K:=J-M;    {3}

```

A definition of M preceding this block is preserved by the block; there are no definitions of M in it. If the program contains definitions for variables $\{A, B, J, K, M\}$, then this block also preserves any definitions for A and B , but not for J and K .

Consider figure 11.14. A definition d (the assignment statement $K := 17$) appears in block B_1 . There are no subsequent definitions of data item K in blocks B_1 or B_2 . We then say that definition d *reaches* block B_3 ; whether it is killed in block B_3 is immaterial. There may also be other paths from B_1 to B_3 along which definition d is killed; we only need one along which the definition is preserved for d to reach B_3 . Block B_1 may also be the same as block B_3 ; we then have a loop through which a definition in block B_1 can reach its own block.

The definition d in block B_1 in figure 11.14 is locally available in B_1 and is available in blocks B_2 and B_3 .

Consider figure 11.15. A use u of K appears in block B_3 , through the statement $M := K + 3$. If no definition of K appears prior to u in block B_3 or anywhere in blocks B_1 and B_2 , then use u is upwards exposed in all three of these blocks.

Next, consider figure 11.16. We have a definition d of data item K in block B_1 , with no subsequent definition in B_1 , B_2 , B_3 , or B_4 . A use of K appears in B_4 . We then say that d is live in blocks B_2 , B_3 , and B_4 . d has clearly reached each of these blocks, and an upwards exposed use of K exists in each of them. Again, the existence of other paths or of definitions in other blocks not shown in the figure is immaterial.

Finally, consider figure 11.17. We have several blocks, containing three definitions and two uses of a variable X . All the control flow paths are shown. We denote a definition of a data item X in a block i by X_i . Sets R_j and L_j for a block j will consist of definitions. Set U_j for block j will consist of the one data item X or be empty.

For example, consider block 3. Only the definition X_2 can reach this block; the definitions in blocks 6 and 7 cannot. Hence R_3 consists of X_2 . Set U_3 contains X , since a use of X appears in block 3 not preceded by a definition. Finally, definition X_2 is live in block 3; none of the other definitions are.

A complete table of reaches, upward exposed uses, and live definitions for the graph of figure 11.17 is given below.

Block i	Reach R_i	Up. Exp. U_i	Live Defs. L_i
1	$\emptyset$	$\emptyset$	$\emptyset$
2	X_2	$\emptyset$	$\emptyset$
3	X_2	X	X_2
4	X_2	X	X_2
5	X_2	X	X_2
6	X_2	$\emptyset$	$\emptyset$
7	X_2, X_6	$\emptyset$	$\emptyset$

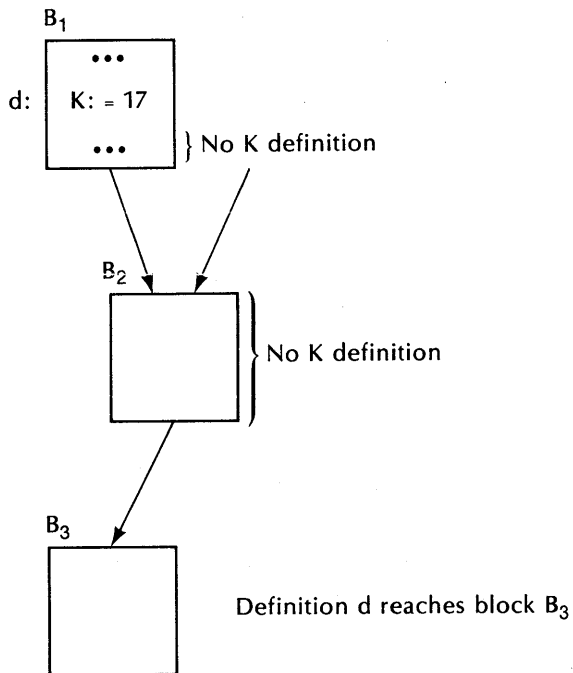


Figure 11.14. Reach of a definition d to block B_3 .

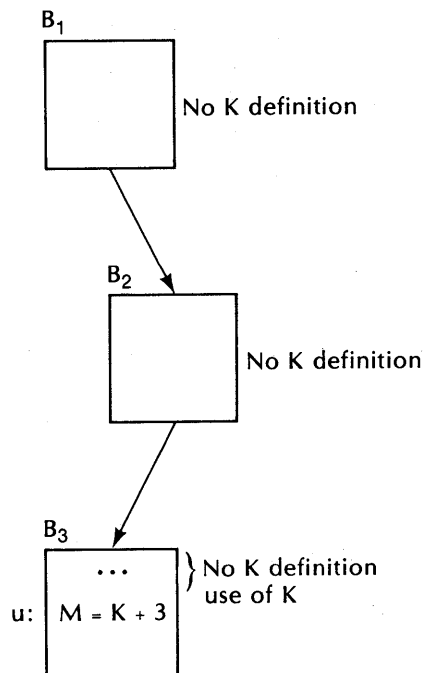


Figure 11.15. Use u of K in B_3 is upwards exposed in all three blocks B_1 , B_2 , B_3 .

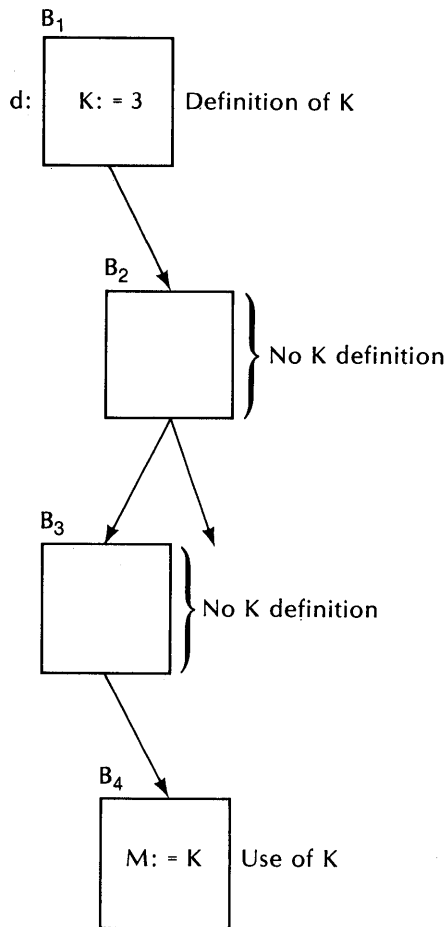


Figure 11.16. Definition of K in block B_1 is live in blocks B_2 , B_3 , B_4 .

Exercises

1. Consider the graph of figure 11.18, consisting of 7 blocks. There are definitions of a data item X in blocks 5 and 7, and uses in blocks 2 and 4. Determine the sets R_i , U_i , and L_i for each block i , $1 \leq i \leq 7$.
2. A Basic program carries a line number on each statement. If such a program contains a computed GOTO statement of the form GO N , where N has been previously evaluated and must evaluate to a line number during program execution, is it possible to construct a data flow graph for the program? What form would it have? Similarly, consider a

Fortran program with a computed GOTO, but such that only some of the statements carry line numbers.

3. Construct a flow graph for the statements of the following program and construct the sets R , U , and L :

```

for G=1 until NOOFGRAPHS-1 do
begin
  if G>1 then
  begin
    for I=FIRSTEDGE(G) until LASTEDGE(G) do
    begin

```

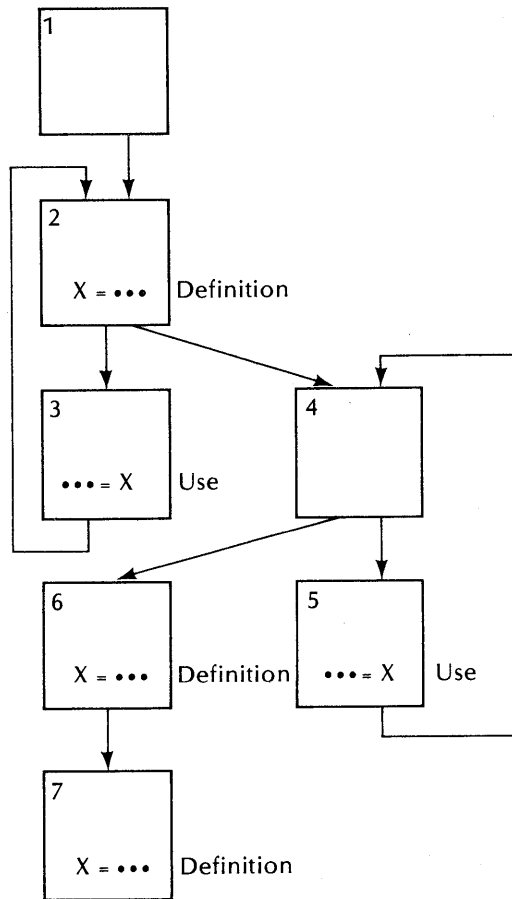


Figure 11.17. A complete flow graph with some definitions and uses of variable X .

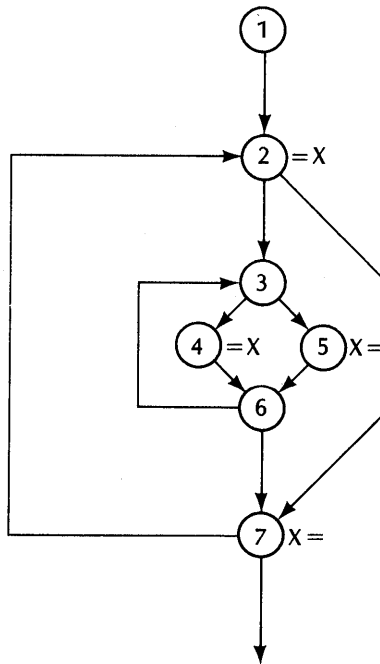


Figure 11.18. An example graph—see exercise 1.

```

PB(I) := P(CORRESEGE(I));
DB(I) := REACH(CORRESHEAD(FROM(I)))
        and PB(I)
        or D(CORRESEGE(I));
end;
for I:=FIRSTNODE(G) until LASTNODE(G) do
begin
  UB(I):=USED(CORRESHEAD(I));
  REACH(I):=REACH(CORRESHEAD(I));
end
end
end;
HEAD:=0;

```

(Note: There are no function calls in this program; all the subscripted references are vector references).

11.5.2. The Basic Data Flow Analysis Method

It should be apparent that the determination of the various sets defined here

can be made by first determining the local sets PB and DB for each block, then exploring all the paths that lead into or out of each of the blocks and the implications of PB and DB on these paths. Unfortunately, path exploration in a directed graph is often a computationally hard problem—the number of operations tends to increase exponentially with the number of nodes in the graph. It turns out that data flow analysis is not computationally hard, provided that a graph reduction technique called *interval ordering* is used. Interval ordering reduces the computational difficulty of flow analysis to one requiring $O(n \log n)$ operations, where n is the number of nodes.

Let us first examine the basic method. It should be apparent that the set of definitions that reach a block B_i is the union of the definitions available from the nodes that are the immediate predecessors (the parents) of B_i . That is,

$$R_i = \cup A_p$$

where the A_p are the available definitions of the blocks B_p that are the parents of block B_i .

Now the set of available definitions A_i for block B_i consists of all those locally available definitions DB_i , together with all those definitions in the program that reach B_i (namely R_i) and that are preserved in B_i . That is,

$$A_i = (R_i \cap PB_i) \cup DB_i$$

These two set equations are sufficient to determine the sets A_i and R_i for all the blocks B_i through a simple algorithm called the *basic reach algorithm*:

Basic Reach Algorithm

The inputs are the PB_i and DB_i for each block B_i in the control flow graph. The outputs are R_i and A_i for each block B_i .

1. Initialize all of the A_i and R_i to the null set.
2. Perform step 3 while there is no change in any R_i or A_i .
3. Apply the following two formulas in succession to the nodes of the graph:

$$R_i = \cup A_p$$

where the A_p are the available definitions of the blocks B_p that are the parents of block B_i .

$$A_i = (R_i \cap PB_i) \cup DB_i.$$

Kildall [1973] has shown that the information in A_i and R_i does stabilize. However, the rate with which the sets stabilize depends critically on the order in which the nodes of the graph are examined; the process can be very time consuming for a poor choice of node ordering.

Note that, to be useful, we must consider a program at least as large as a typical procedure, and build definition and preservation sets for all its definitions and uses. We cannot arbitrarily restrict our interest to some subset of the program. It is therefore important to develop an efficient method of determining these sets.

11.5.3. Intervals

Given a node h , an *interval* $I(h)$ is the maximal, single entry subgraph in which h is the only entry node and in which all closed paths contain h . The unique interval node h is called the *interval head* or simply the *header node*. An interval will consist of an ordered list of nodes; the ordering will be determined by the next algorithm, and will be important in an improved reach algorithm.

Interval Algorithm

1. Let H be a set of header nodes. Initially, let it contain node n_0 , the unique entry node of the flow graph.
2. For every node $h \in H$, fix the interval list $I(h)$ as follows:
 - (a) Put h in $I(h)$ as the first element.
 - (b) Add to $I(h)$ any node all of whose immediate predecessors are already in $I(h)$.
 - (c) Repeat step 2(b) until no more nodes can be added to $I(h)$.
3. Add to H all nodes in G that are not already in H and that are not in $I(h)$, but that have immediate predecessors in $I(h)$. Clearly, a node is added to H the first time any, but not all, of its parents become members of an interval.
4. Select the next unprocessed node in H and repeat steps 2, 3. If no more unprocessed nodes exist in H , the procedure terminates.

The end result is a list of intervals $I(h_1)$, $I(h_2)$, $\dots$, that represents a partition of the nodes of the flow graph.

Example.

Figure 11.19 illustrates the partitioning of a graph into intervals. Interval $I(1)$ consists of only one node 1, since its successor, node 2, has more than node 1 as a parent; there is a path from node 7 to 2 as well as from node 1, and node 7 is not in $I(1)$. Interval $I(2)$ is begun with node 2, and again consists only of that node. (Node 3 can be reached from node 6.) Interval $I(3)$ is begun with node 3, then nodes 4 and 5 can be added, then 6. Node 7 cannot be added, since a path from node 2, not in $I(3)$, exists. Finally, interval $I(7)$ consists of nodes 7 and 8.

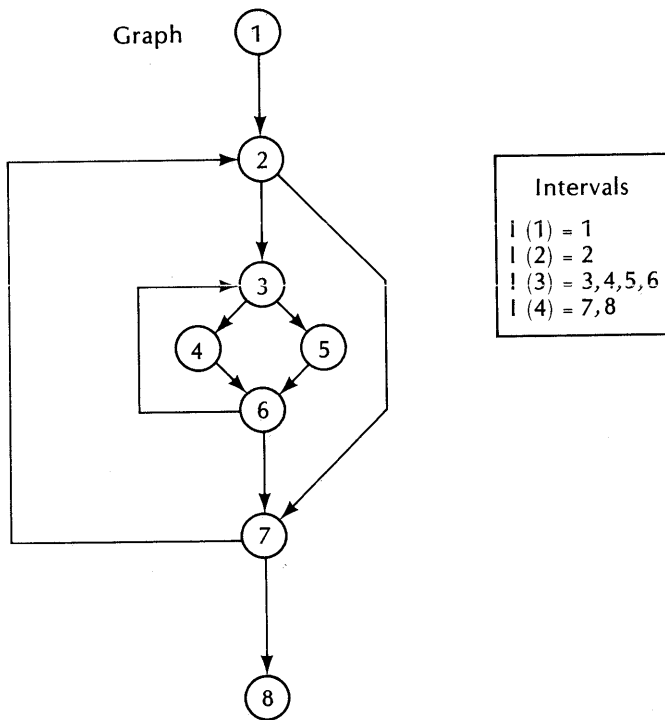


Figure 11.19. Partitioning a graph into intervals.

11.5.4. Higher order intervals

The intervals just determined from the original flow graph G^1 can be expressed as another graph G^2 . We construct G^2 from the intervals of G^1 by merging the nodes within each interval, and discarding all edges connecting two nodes in the same interval. We thus obtain a *second-order interval graph*. This, too, can be reduced by interval partitioning to obtain a third-order, a fourth-order, etc. interval graph.

Eventually, a graph G^i is obtained by interval analysis of a graph G^{i-1} that contains the same number of nodes as G^1 . If G^i contains one node, we say that G^i is *reducible*; otherwise G^i is *irreducible*.

An example of an irreducible graph is given in figure 11.20. Here, $I(1)$ consists of 1 alone, $I(2)$ consists of 2 alone, etc.

It is interesting that programs that consist only of structured control statements (if-then-else, while-do, for, sequential statements) are reducible. Irreducible graphs stem from programs with GOTO statements.

An irreducible graph can be *split* by replicating one or more nodes. For example, if nodes 3 and 4 are duplicated in figure 11.20, we obtain a reducible graph, figure 11.21. Splitting is merely an analysis technique; we are not suggesting that the program code must be duplicated. It also turns out that

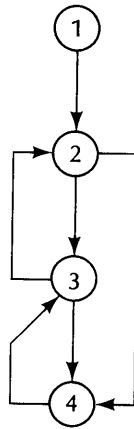


Figure 11.20. An irreducible graph.

even if a control flow graph is irreducible, the analysis that yields sets R and A nevertheless terminates and correctly computes the sets. It will just take more time than a reducible graph with a comparable number of nodes.

The details of the reach algorithm based on intervals may be found in Allen and Cocke's paper (Allen [1976]). The general reach algorithm is in two phases. In the first phase, two items of information are collected by working through the interval graphs in the order $G^1, G^2, \dots$ (the order in which they were constructed) as follows:

1. The definitions defined in the interval and locally available from it. They will become a DB set for the node representing the interval in the next higher order graph.
2. The definitions preserved by the interval; they will become the PB sets for the next higher order graph.

In the second phase, the graphs are worked through in the opposite order of construction; the set of definitions reaching each node is thereby generated.

The essential property of an interval that reduces the length of the analysis is that control that reaches any node within the interval must have passed through the interval header. We say that the header *dominates* the nodes of the interval for this reason. Furthermore, an interval I' that is the successor of an interval I is such that control must pass from some node in I into the header of I' . By considering the interval sets in their generation order, the number of path combinations to be considered are minimized.

It is interesting that Knuth, in his empirical examination of a number of Fortran programs (Knuth [1971]), found that over 90 percent of their control flow graphs were reducible, which implies that a control flow analysis based

on intervals will generally be quite efficient. An analysis of programs written in a more structured language, such as Algol or Pascal would undoubtedly yield an even higher percentage of reducible control flow graphs. An Algol or Pascal program without GOTO statements would be reducible.

11.5.5. Use and Live Information

Given the upwards exposed use U_i and the definitions R_i that reach a block B_i in a program, the live set can readily be determined. However, this set contains only the data items that are live upon entry into a block. It is usually more desirable to know those items that are live on each of the exit edges from a block. For example, register allocation requires live information on exit from a block, rather than on entrance. Rather than retain both entrance and exit information, it is better to retain live information for edges and construct the entry live data item sets as needed; the latter is the union of the former over the entering edges.

We then need the set of definitions A_e available on each edge. Then for a given edge e , incident into node i ,

$$L_e = A_e \cap U_i$$

The algorithm for generating the U_i sets for the graph nodes by the use of the interval sets again requires two phases, the first in interval generation order and the second in opposite order. The first phase can be a part of the reach algorithm. The second phase, however, works backwards through the nodes in each interval.

The upwards exposed uses from each interval is found in the first phase as follows:

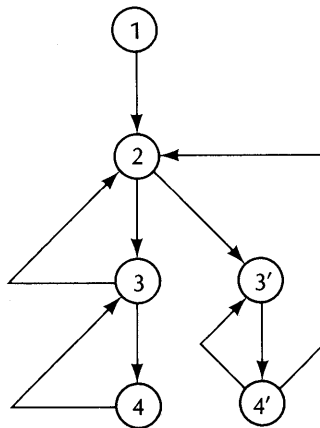


Figure 11.21. Splitting states 3 and 4 in the graph of figure 11.20.

1. Prior to processing each interval h , a set U_h is created and initialized to the upwards exposed uses in that interval.
2. For each node i in the interval ($i = 2, 3, \dots$), U_h is updated with the locally exposed uses UB_i in i that can be preserved along some path from h to i , which is done by forming the set

$$(\cup P_p) \cap UB_i$$

where the union is over the sets P_p ; P_p is the set of data items preserved on input edge p to node i . Its computation is part of the basic reach algorithm.

The complete algorithms and a procedure in PL/I may be found in Allen [1976].

11.5.6. The Interval-Based Reach Algorithm

Inputs

1. The ordered set of graphs ($G^1, G^2, \dots, G^n$) determined by interval analysis.
2. The intervals in each graph with their nodes given in interval process order.
3. The definitions *defined* and *preserved* on each edge in the first order graph. These are expressed in the DB and PB sets.

Outputs

1. A set R of the definitions that reach each node.
2. A set A of the definitions available on each edge.

Phase I

1. For each graph G^g in the order $G^1, G^2, \dots, G^{n-1}$, perform steps 2 and 3 that follow:

2. If the current graph is not G^1 then initialize the PB and DB sets for the edges of the graph. PB and DB are initialized by first identifying the edge in G^{g-1} to which each edge in G^g corresponds (these will be interval exit edges). Then, using the information generated during step 3 for G^{g-1} , for each edge i in G^g with corresponding exit edge x from interval with head h in G^{g-1} , set:

(a) $PB_i := P_x$ and

(b) $DB_i := (R_h \cap P_x) \cup D_x$

3. For each exit edge of each interval in G^g , determine P , the definitions preserved on some path through the interval to the exit, and D , the definitions in the interval that may be available on the exit. These definitions are found by finding P and D for each edge in the interval as follows:

(a) For each exit edge i of the header node:

$$P_i := PB_i$$

$$D_i := DB_i$$

(b) For each exit edge i of each node j ($j = 2, 3, \dots$) in interval order:

$$P_i := (\cup P_p) \cap PB_i$$

$$D_i := ((\cup D_p) \cap PB_i) \cup DB_i$$

where the union over P_p and D_p is for all p input edges to node j .

While processing an interval, determine the set of definitions R_h that can reach the interval head h , from inside the interval by

$$R_h := \cup D_l$$

for all interval edges l which enter h . If there are none set R_h to $\emptyset$.

Between phases I and II, the R vector for the single node in the n 'th order derived graph is initiated: $R_1 = \emptyset$ or whatever set of definitions is known to reach the program from outside.

Phase II

1. For each graph G^g in the order $G^{n-1}, \dots, G^2, G^1$, perform the following steps 2 and 3.

2. For each node i in G^{g+1} form $R_h := R_h \cup R_i$, where h is the head of the interval in G^g which i represents in G^{g+1} .

3. For each interval process the nodes in interval order determining the definitions reaching each node and available on each node exit edge as follows:

(a) For each exit edge i of the header node h

$$A_i := (R_h \cap PB_i) \cup DB_i$$

(b) For each node j ($j=2, 3, \dots$) in interval order first form

$$R_j := \cup A_p, \text{ for all input edges } p \text{ to } j$$

then for each exit edge i of j form

$$A_i := (R_j \cap PB_i) \cup DB_i$$

Exercise

Consider the graph G^1 and its higher order interval graphs shown in figure 11.22. There are two variables, X and Y , and three definitions X_1 , X_4 , and Y_3 .

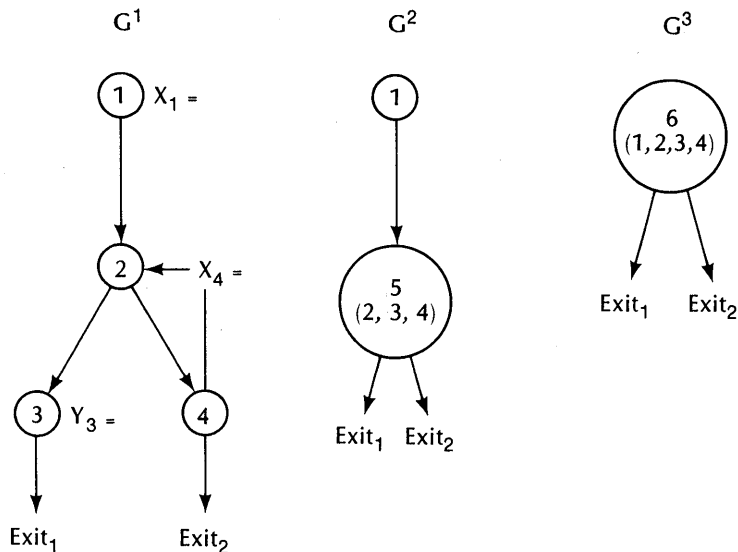


Figure 11.22. A graph and its higher-order intervals.

- (a) Show that figure 11.23 expresses DB and PB for the graphs.
- (b) Show that figure 11.24 expresses the reach and availability sets R and A after the second phase of the reach algorithm.

	Edge	DB	PB
G^1	1-2	X_1	$X_4 Y_3$
	2-3	0	$X_1 X_4 Y_3$
	2-4	0	$X_1 X_4 Y_3$
	3- e_1	Y_3	$X_1 X_4$
	4-2	X_4	$X_1 Y_3$
	4- e_2	X_4	$X_1 Y_3$
G^2	1-5	X_1	$X_4 Y_3$
	5- e_1	$X_4 Y_3$	$X_1 X_4$
	5- e_2	X_4	Y_3
G^3	6- e_1	$X_1 X_4 Y_3$	$\emptyset$
	6- e_2	$X_1 X_4$	Y_3

Figure 11.23. DB and PB for the graph of figure 11.22.

	Node	Reach	Edge	Available
G^3	6	$\emptyset$	6-e ₁	$X_1 X_4 Y_3$
			6-e ₂	X_4
G^2	1	$\emptyset$	1-5	X_1
	5	X_1	5-e ₁	$X_1 X_4 Y_3$
			5-e ₂	X_4
G^1	1	$\emptyset$	1-2	X_1
	2	$X_1 X_4$	2-3	$X_1 X_4$
	3	$X_1 X_1$	2-4	$X_1 X_4$
	4	$X_1 X_4$	3-e ₁	$X_1 X_4 Y_3$
			4-2	$X_1 X_4$
			4-e ₂	$X_1 X_4$

Figure 11.24. REACH and AVAILABILITY sets for the graph of figure 11.22.

11.5.7. Applications of Data Flow Information

We are now in a position to outline a number of useful optimizations based on the information developed in a data flow analysis.

Useless definitions. A definition d is *useless* if the value assigned to the data variable X in d is never subsequently required by any reference. The definition d may be removed with no effect on the program. Let d reside in a block B_i . Now d will be useless if:

1. Definition d is live on the B_i block exit, i.e., d is in L_i .
2. No upwards exposed use of d exists on exit from B_i .

Let U' be the union of the sets L_e for each edge e leaving block B_i . If d is not in U' , then d is useless. Set U' is a *useful* set of definitions.

Uninitialized Variable Use. If a use u is upwards exposed in the "start" block B_0 , then a path from the program origin to u exists that does not contain a definition of the use data variable. Clearly, a test for potentially uninitialized variables in a program is that $U_0 = \emptyset$. We say "potentially", because data flow analysis does not consider the logic of program branching, and it may happen that the path to the uninitialized variable use cannot be followed during program execution.

Basic Block Input and Output Variables. An input variable X of a block B is such that a use u of X appears in B , and u is upwards exposed in B . As we have seen earlier, we do not need a data flow analysis to identify input variables of a basic block.

An output variable X of a basic block B must appear in B (use or definition)

and is live on exit from B. The *live* set of course derives from data flow analysis.

Loop Code Movement. Suppose that a loop is identified, as indicated in figure 11.25. B1 and B2 are some collections of code with single entries and exits; these are not necessarily basic blocks. B3 is a basic block containing a statement S of interest. T is a test for loop exit, and contains only uses. Statement S has the form

$$X \leftarrow \theta Y_1 Y_2 \dots Y_r$$

where θ is an r -ary operation, the X receives the result, and the Y_i are the operands.

It is sometimes possible to move S from B3 into block B1 without affecting the program. If this can be done, we will have one execution of S in the optimized program for every n executions in the original program, where n is the number of loop iterations. Let us develop necessary and sufficient conditions for such a code movement.

S can be moved to the bottom of B1 only if no reachable definitions of any of the Y_i or X exist in B2, other than in S. Since block B2 is not necessarily basic, we need the results of a data flow analysis.

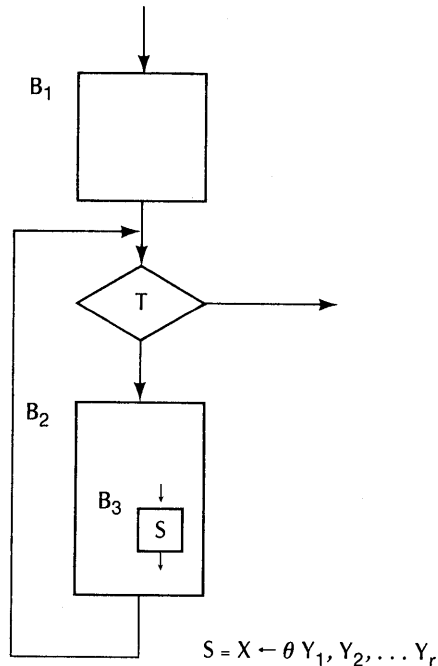


Figure 11.25. Code movement of a statement S from block B₂ to block B₁.

Clearly, all of the uses u_i ($1 \leq i \leq r$) corresponding to the operands Y_i in S must be in the upwards exposed use set U_2 of block B_2 . They must also be live at the block B_2 exit. These conditions guarantee that no loop path can change any of the Y variables.

The definition of X in statement S must be live at block entry B_2 (therefore also at exit of B_2) and at block entry B_3 . This requirement clearly precludes any other definitions of X in B_2 (except in S and unreachable definitions); such a definition would either kill the S definition at block B_2 exit or at block B_3 entry.

11.6. Bibliographic Notes

Nakata [1967] and Meyers [1965] developed the optimal algorithm for the generation of code from an expression tree; their algorithm yields the minimum number of registers for commutative operators only. Sethi and Ullman (Sethi [1970]) restated their algorithm, and showed that it is optimal for noncommutative and nonassociative operators as well as commutative and associative-commutative operators.

The notion of collecting complicated nodes to the left of simple nodes for a machine with noncommutative instructions that operate between a register and memory can be traced to an early paper by Floyd [1961].

The discussion of blocks, block statements, and renaming is principally drawn from Aho [1972b]. A presentation similar to ours may be found in Aho and Ullman (Aho [1972]), chapter 11.

The data flow analysis material may be found in Allen and Cocke's paper (Allen [1976]).

Beatty [1974] describes an algorithm that assigns registers in a highly optimal fashion, and he considers both the local domain of a block and its global environment.

Bruno and Sethi (Bruno [1976]) and Aho, Johnson, and Ullman (Aho [1977b]) show that the problem of code optimization for a general class of single and multiregister machines, given a DAG with common subexpression subsumption, is NP-complete, i.e., has a computational complexity that is exponential in the size of the DAG. The significance of this result is that optimal solutions can be demonstrated only for rather small DAGs.

Cocke and Kennedy (Cocke [1977]) has given a simple algorithm that performs reduction of operator strength by moving certain statements out of a FOR loop. A general discussion of loop strength reduction, identification of induction variables, etc. is given in Aho and Ullman (Aho [1972a]), chapter 11.

ERROR RECOVERY

12.1. Introduction

Since the source for a compiler or interpreter is generally prepared by a person, it is likely to contain errors. The compiler must therefore have a reasonable mechanism for dealing with errors. A set of objectives of a satisfactory error recovery system are

1. *Report* the error, indicating its location.
2. *Diagnose* the error, as an aid to its correction.
3. *Recover* from the error such that subsequent errors are detected and spurious error reports are not generated.

We shall see that these objectives can be approached, but not completely met. Objective (1) can always be met, provided that the error has not been masked by the recovery from a preceding error. Objectives (2) and (3) are rather difficult to meet. Effective diagnosis of syntax errors is difficult, and most error recovery systems tend to generate spurious error reports after recovery from some error.

The *correction* of an error, such that the corrected program will execute the programmer's intended algorithm correctly, is not feasible with the commonly used programming languages. It would only be feasible if the program's algorithm were to be expressed in two different, but equivalent ways. With only one statement of the algorithm, and that statement erroneous, a compiler can only patch over the error sufficiently well that it can continue to parse the remaining portion of the program.

The compiler cannot in general determine the error made by the programmer; it sees only an *error symptom*, some condition reached during the parsing process such that the parse cannot continue. An error may or may not immediately produce an error symptom; however, every error symptom is the result of some error. For example, the following Fortran statement contains one too many left parentheses:

$$A = B * (C - (D * F)$$

↑

The error symptom is a missing right parenthesis, detected at the end of the line, as indicated by the arrow $\uparrow$. However, the error could be earlier in the line; perhaps the programmer intended to write

$$A = B * (C - D * F)$$

and entered an extra left parenthesis by mistake. The compiler cannot correct the error in general, it can only report an error symptom, and the symptom is often several tokens past the error. We shall often use the term *error* to refer to an *error symptom*.

Error Diagnosis

A reasonable error diagnosis would be an indication of the location of the error, and a brief message indicating the kind of error encountered. Some errors cannot be detected until the end of a block or program; it is especially important that these be clear, and that as much information as possible be provided by the compiler.

The error messages generated during a compilation may be collected and reported at the end of the compilation, or may be interspersed with the source listing. An interspersed listing may be the only possible choice if storage capacity is limited. Otherwise, the choice is a matter of user preference.

If interspersed messages are used, each one should contain a reference to the previous one, and a message at the end of the program should refer to the last message. Otherwise, an isolated error message is easily overlooked or hard to find in a large listing.

An error summary at the end of a listing is important in order for a programmer to quickly determine whether his program is free of errors, and if it is not, to be able to locate each of the errors.

Kinds of Error

There are three major classes of error, depending on whether they are detected in the lexical analyzer (possibly including a macro generator), the parser, or the semantic checking sections of the compiler. We shall call these *scanner*, *syntax* and *semantic* errors, respectively. A fourth class might be established for a developmental compiler: compiler errors or *bugs*, presumably never seen by an ultimate user, but useful during development to warn of compiler states that should not have occurred.

A *scanner* error results from some problem in identifying a token, or reaching an end of file in the middle of a token, or failure to open or read the source file. A diagnostic message can say nothing about the error's meaning, but must be confined to the specific source problem found. Recovery from a scanner error is relatively simple: if the file end is found within a token,

terminate the program. If an invalid character is found, reject it and start over with the next token. Tokens that require some parsing, such as floating-point number tokens, will require attention to syntax errors whose nature should be clear from the state reached during the scan. (The scanner is a finite-state automaton.)

An unclosed comment or quoted string cannot usually be detected by the scanner; its effect must be detected by the parser.

For example, consider the following program fragment:

```
MOVE 'SAM'S FRIEND' TO BUFFER;
MOVE 'FREDDY AND MARY' TO BUFFER;
```

The quote mark just following SAM in the first line causes the two lines to be divided into tokens as follows:

```
token: MOVE
string: SAM
tokens: S FRIEND
string: TO BUFFER; MOVE
tokens: FREDDY AND MARY
string: TO BUFFER;
(etc.)
```

The remaining program is inverted—quoted strings will be regarded as program material and the program material will be regarded as quoted strings. The scanner will in general be unaware of this until an end of file or end of record is detected inside a quoted string. However, the parser will be reporting a large number of syntax errors.

A macro error usually cannot be detected until the compiler attempts to parse it. Then a simple macro error is often magnified into a voluminous error report.

A *syntax error* is detected by the parser in a READ operation. Some token x , called the *error token*, is incompatible with the state of the parser at the point at which x is to be scanned. In an LR(1) parser, x is detected in a read state P such that no transition from P on token x exists. In an LL(1) parser, x is detected through a mismatch between x and a terminal stack top token, or through failure to accept x on a nonterminal stack top in the transition table.

Error recovery in line-oriented source can be very easy. Line-oriented source is subdivided into lines, such that the parser may (for the most part) start over from a known state on each line. Thus when a syntax error is detected in some line, recovery is simply a matter of skipping the rest of the line and starting over on the following line. Whether the source is line-oriented or not depends on the language. Fortran, Basic and APL are examples of line-oriented source. Algol, Pascal, and PL/I are not.

Line-oriented recovery doesn't always work properly. For example, in Fortran, a line can contain several declarations. If some of these are skipped because of a syntax error earlier in the line, a number of semantic errors will likely be reported later in the program. Also, if a syntax error occurs in the first statement of a subroutine, special attention must be paid to getting the compiler into a state that will accept the subroutine statements.

Error recovery in free-form source, for example Algol or Pascal, is more difficult. There are no distinctive line boundaries to aid recovery. However, certain tokens such as semicolon or END could be used in a recovery. PL/I in particular has the rule that every statement is terminated by a semicolon, and the semicolon has no other purpose. Hence PL/I error recovery can be achieved by skipping source through the next semicolon.

A better strategy for free-form languages is to attempt to patch the region of the error, by dropping and/or inserting tokens such that the sentence is brought into the language. We shall discuss such approaches in section 12.3.

A *semantic error* is any error symptom in a section of source that is free of syntax errors. A semantic error is detected through some special test called out in an APPLY action. Since the parse has been successful, the nature of the error is usually very clear, and a specific diagnostic message can be easily formulated. Section 12.2 contains more details.

Semantic errors arise out of a failure to describe a programming language through its grammar alone. (However, some languages, notably APL and Algol 68, are fully defined through their grammars.) For example, the attributes of a declaration may have to satisfy some relation that would be difficult or impossible to define in a context-free grammar, or the sizes of certain literals may be restricted in some fashion.

Effect of an Error on the Compiler

Suppose that an error is detected in a source program. What changes in the compiler should take place? The answer to this question depends on the kinds of errors that are to be subsequently detected, and on the nature of the code generation process.

A block-structured, free-form program containing an error cannot usually be repaired by the compiler to the complete satisfaction of the programmer. Any repair made by the compiler is for the purpose of continuing to check for errors, not to create a "correct" program. Hence, once an error is detected, there is no point in carrying out any compiler operations beyond those needed for further error detection.

One question in error recovery design is whether semantic checking should continue after a syntax error. One important semantic check is for undeclared and misused identifiers. Should these be suppressed upon the first error or not? There are two schools of thought. If semantic checking is discontinued, then semantic errors past a syntax error are unreported. If semantic checking

is continued, it often happens that the recovery from a syntax error will cause the appearance of spurious semantic errors. Syntax error recovery in a declaration is especially likely to cause spurious semantic error messages.

It is certainly much easier to design the compiler such that all code generation and semantics checking stop upon the first syntax error. The compiler will also run faster in its "syntax-checking" mode, being relieved of an appreciable number of synthesis operations. Whether this will be appreciated by a user is not clear.

Most commercial compilers provide a limited amount of semantic checking, at least the checking that normally would be made in the pass in which an error was found. Users of compilers seem to accept the possibility of spurious error messages, particularly when the compiler reports the sort of changes it made in the user program during error recovery. Serious errors found in one pass of a multipass compiler are usually sufficient reason to end the compilation on that pass. However, all the syntax errors are found on the first pass. Since semantic errors are usually easy to repair, it is possible to continue with all the passes, given no syntax errors in the first pass.

Some compilers are designed to make incremental adjustments to a code file through *partial compilation*. For example, one procedure out of a set of procedures may be selected for a recompilation. This approach to compilation reduces compilation time during program development and facilitates writing procedures in different languages that are intended for the same program. If this is the case, the compiler is given a code file that it may update, but should not destroy. It is then important that, upon an error, the code file be closed without destroying its validity. Otherwise, a complete compilation to reconstruct the damaged code file would be required after each syntax error recovery.

A more difficult requirement for a compiler can be expressed as follows: Accept a code file and update it with the code for every procedure that can be correctly compiled, but do not update it for procedures that contain errors. Although this appears to be an impossible requirement, in fact it can be achieved. Essentially, a decision is made at the end of a procedure whether to update the code file or not. An error found in the procedure, or in a covering procedure, is sufficient to not update the file. It is still possible to damage the code file on certain kinds of syntax error, but our experience indicates that, given a reasonable error recovery strategy, the file is rarely damaged. Clearly, if any procedure code has been written to the code file prior to detection of an error, there must be some means of marking it invalid, or removing it.

12.2. Semantic Errors

A *semantic error* is any error detected in an APPLY operation on source that is free of scanner or syntax errors. Semantic tests are usually necessary,